WESTERN SYDNEY UNIVERSITY

School of Computer, Data and Mathematical Sciences

COMP 7006 Data Science

Computer Based Assignment – PART B

Q4, 2024

Complete your details in this section.	
STUDENT ID:	22119424
STUDENT FIRST NAME:	AESWARYA
STUDENT SURNAME:	KRISHNAKUMAR

QUESTIONS FORMAT:	Word processed document in PDF format; logically presenting answers to each question incorporating R outputs including graphs and charts.
TOTAL MARKS:	**35 Marks (**30 for report + 5 for organisation, presentation of R codes and outputs in Appendix**)**
UNIT CO-ORDINATOR:	Dr. Liwan Liyanage
LECTURERS:	Prathayne Nanthakumaran Lakmini Wijesekara
TUTORS:	Prathayne Nanthakumaran Lakmini Wijesekara Irangi Samarakoon
TOTAL PAGES:	54

INSTRUCTIONS

Please note that you are expected to answer the questions clearly in this document. Use the template included where relevant to answer. Give the R outputs, comments, and discussion clearly and logically.

Attach all the R commands and outputs in the Appendix.

Write the resulting model equation to the relevant questions with R outputs, especially when you are asked to give the resultant model whenever applicable.

Once completed submit the answer scripts as a **PDF** via the link within vUWS site.

Please note that **5 Marks** is allocated for organization, reasoning, logical flow, and the inclusion of all correct R codes and outputs in the Appendix.

NOTE: Academic misconduct will result in serious consequences. Please refer to the subject outline for more details.

SCENARIO

An insurance company aims to predict the amount of property damage claims resulting from bushfires to ensure they maintain sufficient financial reserves for post-disaster recovery. To achieve this, they have collected comprehensive data related to location, property characteristics, and meteorological conditions, as detailed below. This data will be used to create accurate predictions of property damage claims. Accurate prediction is essential for the company to manage its financial resources effectively, fulfill client expectations, and maintain operational stability in the face of disasters.

Data Description:

Variables	Description
Fire_Intensity	The intensity of the fire measured in kilowatts per metre (kW/m), representing the heat output during the fire event.
Distance_from_Fire	The distance of the building from the fire's origin in kilometres (km)
Building_Age	The number of years since the building was constructed
Property_Value	The estimated value of the property in hundred thousand Australian Dollars ($\times 10^5$ AUD)
Population_Density	The number of people living per square kilometre in the area where the building is located
Emergency_Response_Time	The time taken for emergency services to respond, in minutes

Mitigation_Measures	The number of proactive steps taken to reduce fire risk, such as fireproofing materials, which contribute to minimizing fire damage.
Construction_Quality	A categorical variable assessing the quality of the building, classified as either "Good" or "Bad," based on materials, design, and compliance with safety standards.
Insurance_Coverage	The type of insurance coverage the property owner has, categorized as "Fully," "Partially," or "None," reflecting the level of financial protection available.
Wind_Speed	The average wind speed recorded during the bushfire event in ms^{-1}
Humidity	The average relative humidity during the bushfire in percentage (%)
Damage_Claims	The monetary value of damage claims submitted after the fire, in hundred thousand Australian Dollars (($\times 10^5$AUD), indicating the financial cost of the fire damage.

* Please note that this is a simulated data generated to resemble the real-world data for the purpose of this assignment.

Consider the scenario described and the data set provided [***BushFireData.csv***] to answer the following questions.

1. Build a support vector machine to correctly classify whether the damage claim would be greater than 7.5 hundred thousand AUD or not. Clearly outline and explain each step of the process involved. [*Hint: Consider all the three Kernels with different values of parameters*] **(9 Marks)**

Step 1: Load the dataset and check the structure of the dataset using str( ) in order to identify the data types

```
bushfire_data <- read.csv("BushFireData.csv")
```

Step 2: Convert the categorical variables such as Construction quality and insurance Coverage to factor.

```
bushfire_data$Construction_Quality <- as.factor(bushfire_data$Construction_Quality)

bushfire_data$Insurance_Coverage <- as.factor(bushfire_data$Insurance_Coverage)
```

Step 3: Create the binary response variable for classification and convert damage high to factor.

```
bushfire_data$Damage_High <- ifelse(bushfire_data$Damage_Claims > 7.5, 1, 0)
bushfire_data$Damage_High <- as.factor(bushfire_data$Damage_High)
```

```r
str(bushfire_data)
```

```
## 'data.frame':    600 obs. of  14 variables:
##  $ ClaimID                : chr  "DQTA26862" "RDAQ32534" "WTLA41817"
"YHZL16317" ...
##  $ Fire_Intensity         : num  5738 12248 7317 13479 14226 ...
##  $ Distance_from_Fire     : num  5.57 5.49 5.12 4.89 6.13 ...
##  $ Building_Age           : int  19 27 16 30 21 27 24 24 36 30 ...
##  $ Property_Value         : num  9.76 5.89 3.98 4.91 9.31 ...
##  $ Population_Density     : int  520 433 634 576 644 476 500 591 487
441 ...
##  $ Emergency_Response_Time: int  16 18 17 14 12 10 16 19 15 18 ...
##  $ Mitigation_Measures    : int  3 3 4 2 2 1 5 7 2 3 ...
##  $ Construction_Quality   : Factor w/ 2 levels "Bad","Good": 2 2 2 2
2 2 2 2 2 2 ...
##  $ Insurance_Coverage     : Factor w/ 3 levels "Fully","None",..: 1 2
3 3 2 1 3 1 1 3 ...
##  $ Wind_Speed             : num  24.9 21 25.3 22.3 37.2 31.8 27.8 22.
5 32.2 20.6 ...
##  $ Humidity               : num  35.3 28.2 44.5 53.8 73.8 ...
##  $ Damage_Claims          : num  7.02 7.45 4.3 7.78 9.46 4.54 7.14 7.
73 6.03 5.37 ...
##  $ Damage_High            : Factor w/ 2 levels "0","1": 1 1 1 2 2 1 1
2 1 1 ...
```

Step 4: Checking the values of Damage High in order to identify whether its binary or not.

```r
head(bushfire_data$Damage_High)
```

```
## [1] 0 0 0 1 1 0
```

```
## Levels: 0 1
```

- As we can see I have created new binary variable and converted it into factor.

Step 5: Using the full dataset to understand the process, fitting a model with cost = 1. So first filter the dataset and remove irrelevant columns.

```r
bushfire <- bushfire_data %>% select(-ClaimID, -Damage_Claims)

str(bushfire)
```

```
## 'data.frame':    600 obs. of  12 variables:
##  $ Fire_Intensity         : num  5738 12248 7317 13479 14226 ...
##  $ Distance_from_Fire     : num  5.57 5.49 5.12 4.89 6.13 ...
##  $ Building_Age           : int  19 27 16 30 21 27 24 24 36 30 ...
##  $ Property_Value         : num  9.76 5.89 3.98 4.91 9.31 ...
##  $ Population_Density     : int  520 433 634 576 644 476 500 591 487
441 ...
##  $ Emergency_Response_Time: int  16 18 17 14 12 10 16 19 15 18 ...
##  $ Mitigation_Measures    : int  3 3 4 2 2 1 5 7 2 3 ...
##  $ Construction_Quality   : Factor w/ 2 levels "Bad","Good": 2 2 2 2
2 2 2 2 2 2 ...
##  $ Insurance_Coverage     : Factor w/ 3 levels "Fully","None",..: 1 2
3 3 2 1 3 1 1 3 ...
```

```
##  $ Wind_Speed             : num  24.9 21 25.3 22.3 37.2 31.8 27.8 22.
5 32.2 20.6 ...
##  $ Humidity               : num  35.3 28.2 44.5 53.8 73.8 ...
##  $ Damage_High            : Factor w/ 2 levels "0","1": 1 1 1 2 2 1 1
2 1 1 ...
```

Step 6: Building and tuning the linear kernel SVM Model for classification.

```
svm_fit1=svm(Damage_High ~.,data = bushfire, kernel="linear", cost=1,sca
le=TRUE)

 summary(svm_fit1)

##
## Call:
## svm(formula = Damage_High ~ ., data = bushfire, kernel = "linear",
##     cost = 1, scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  1
##
## Number of Support Vectors:  159
##
##  ( 80 79 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

- The model fitted was with the Linear kernel function with the cost of 1 and there were 159 support vectors. 80 support vectors are from one class and 79 support vectors are from the other class.

```
set.seed(1)
tune_out = tune(svm, Damage_High ~ ., data = bushfire, kernel = "linear"
, ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000)),scale=TRUE
)

 summary(tune_out)

##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost
##   0.1
##
```

```
## - best performance: 0.1016667
##
## - Detailed performance results:
##    cost     error dispersion
## 1 1e-03 0.2333333 0.04230985
## 2 1e-02 0.1583333 0.04795188
## 3 1e-01 0.1016667 0.04743416
## 4 1e+00 0.1183333 0.05238155
## 5 1e+01 0.1200000 0.04766136
## 6 1e+02 0.1200000 0.04766136
## 7 1e+03 0.1200000 0.04766136
```

Step 7: Finding out the best parameters.

```
tune_out$best.parameters

##   cost
## 3  0.1
```

Step 8: Finding the best model of linear kernel.

```
best_model1 <- tune_out$best.model
summary(best_model1)

##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = bushfire,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000)),
##     kernel = "linear", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  0.1
##
## Number of Support Vectors:  194
##
##  ( 99 95 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

- From the above output, it can be clearly seen that the optimal value of cost when using 10- fold cross validation is 1 since the error 0.1016667 is minimum when the cost is 0.1. Thus, the optimal model using a linear kernel function can be obtained when cost is 0.1. The best model thus obtained consists of 194 support vectors with 99 support vectors from one class and 95 support vectors from the other class.

Step 9: Now using polynomial basis kernel, with different values of degree and cost.

```
set.seed(1)
 tune_out2 = tune(svm, Damage_High ~ ., data = bushfire, kernel =
"polynomial",
 ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), d =
c(2:5)),
 scale=TRUE)
```

```
summary(tune_out2)

##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost d
##    10 2
##
## - best performance: 0.125
##
## - Detailed performance results:
##     cost d     error dispersion
## 1  1e-03 2 0.2333333 0.04230985
## 2  1e-02 2 0.2333333 0.04230985
## 3  1e-01 2 0.2233333 0.04021547
## 4  1e+00 2 0.1533333 0.03832931
## 5  1e+01 2 0.1250000 0.03068156
## 6  1e+02 2 0.1666667 0.03928371
## 7  1e+03 2 0.1766667 0.03784308
## 8  1e-03 3 0.2333333 0.04230985
## 9  1e-02 3 0.2333333 0.04230985
## 10 1e-01 3 0.2183333 0.04406084
## 11 1e+00 3 0.1550000 0.03853009
## 12 1e+01 3 0.1583333 0.03356402
## 13 1e+02 3 0.1683333 0.04116363
## 14 1e+03 3 0.2000000 0.03600411
## 15 1e-03 4 0.2333333 0.04230985
## 16 1e-02 4 0.2333333 0.04230985
## 17 1e-01 4 0.2200000 0.04430534
## 18 1e+00 4 0.1716667 0.04377975
## 19 1e+01 4 0.1600000 0.03944053
## 20 1e+02 4 0.1766667 0.05676462
## 21 1e+03 4 0.1850000 0.05178898
## 22 1e-03 5 0.2333333 0.04230985
## 23 1e-02 5 0.2333333 0.04230985
## 24 1e-01 5 0.2200000 0.04430534
## 25 1e+00 5 0.1850000 0.04996913
## 26 1e+01 5 0.1600000 0.03617516
```

```
## 27 1e+02 5 0.1666667 0.05030770
## 28 1e+03 5 0.1750000 0.04461772
```

Step 10: Finding out the best parameters

```
tune_out2$best.parameters

##   cost d
## 5   10 2
```

Step 11: Finding the best model of polynomial kernal

```
best_model2 = tune_out2$best.model
 summary(best_model2)

##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = bushfire,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), d = c
(2:5)),
##     kernel = "polynomial", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  polynomial
##        cost:  10
##      degree:  2
##      coef.0:  0
##
## Number of Support Vectors:  187
##
##  ( 103 84 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

- It is clear from the above output that when using polynomial kernel function, the best parameters are cost = 10 and degree = 2 since the error (0.125) is minimum at these values. And the best model found using a polynomial kernel function has 187 support vectors with 103 support vectors in one class and 84 support vectors in the other class.

Step 12: Building and tuning the Radial kernel SVM Model for classification

```
set.seed(1)
 tune_out3 = tune(svm, Damage_High ~ ., data = bushfire, kernel = "radia
l",
 ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), gamma = c(0
.5, 1, 2, 3,4)), scale=TRUE)
```

```r
summary(tune_out3)

##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost gamma
##    10   0.5
##
## - best performance: 0.2
##
## - Detailed performance results:
##     cost gamma     error dispersion
## 1  1e-03   0.5 0.2333333 0.04230985
## 2  1e-02   0.5 0.2333333 0.04230985
## 3  1e-01   0.5 0.2333333 0.04230985
## 4  1e+00   0.5 0.2200000 0.04567734
## 5  1e+01   0.5 0.2000000 0.05270463
## 6  1e+02   0.5 0.2000000 0.05270463
## 7  1e+03   0.5 0.2000000 0.05270463
## 8  1e-03   1.0 0.2333333 0.04230985
## 9  1e-02   1.0 0.2333333 0.04230985
## 10 1e-01   1.0 0.2333333 0.04230985
## 11 1e+00   1.0 0.2333333 0.04230985
## 12 1e+01   1.0 0.2350000 0.04335470
## 13 1e+02   1.0 0.2350000 0.04335470
## 14 1e+03   1.0 0.2350000 0.04335470
## 15 1e-03   2.0 0.2333333 0.04230985
## 16 1e-02   2.0 0.2333333 0.04230985
## 17 1e-01   2.0 0.2333333 0.04230985
## 18 1e+00   2.0 0.2333333 0.04230985
## 19 1e+01   2.0 0.2333333 0.04230985
## 20 1e+02   2.0 0.2333333 0.04230985
## 21 1e+03   2.0 0.2333333 0.04230985
## 22 1e-03   3.0 0.2333333 0.04230985
## 23 1e-02   3.0 0.2333333 0.04230985
## 24 1e-01   3.0 0.2333333 0.04230985
## 25 1e+00   3.0 0.2333333 0.04230985
## 26 1e+01   3.0 0.2333333 0.04230985
## 27 1e+02   3.0 0.2333333 0.04230985
## 28 1e+03   3.0 0.2333333 0.04230985
## 29 1e-03   4.0 0.2333333 0.04230985
## 30 1e-02   4.0 0.2333333 0.04230985
## 31 1e-01   4.0 0.2333333 0.04230985
## 32 1e+00   4.0 0.2333333 0.04230985
## 33 1e+01   4.0 0.2333333 0.04230985
## 34 1e+02   4.0 0.2333333 0.04230985
## 35 1e+03   4.0 0.2333333 0.04230985
```

Step 13: finding the best parameter.

```r
tune_out3$best.parameters
```

```
##   cost gamma
## 5   10   0.5
```

Step 14: Finding the best model of radial.

```
best_model3 = tune_out3$best.model
summary(best_model3)

##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = bushfire,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), gamma
= c(0.5,
##         1, 2, 3, 4)), kernel = "radial", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  10
##
## Number of Support Vectors:  544
##
##  ( 405 139 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

- It can be seen clearly that when using radial kernel function, the best parameters are cost = 10 and gamma = 0.5 since the error (0.2) is minimum at this value. The optimal model thus obtained when using a radial kernel function has 544 support vectors with 405 support vectors in one class and 139 in other class.

Step 15: Dividing the dataset into training and testing. And using the test set to assess the accuracy of the model and training set to fit the model.

```
set.seed(1)
trid <- sample(1:nrow(bushfire), nrow(bushfire)*0.7)
trainset <- bushfire[trid,]
testset <- bushfire[-trid,]
```

Step 16: Tuning the linear kernel using trainset and finding the best model and best parameter as well using the code below

```
set.seed(1)
tune_1=tune(svm, Damage_High ~., data = trainset, kernel="linear", scale
= TRUE, ranges = list(cost=c(0.001, 0.01, 0.1, 1, 10, 100, 1000)))

summary(tune_1)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost
##   0.1
##
## - best performance: 0.08571429
##
## - Detailed performance results:
##    cost      error dispersion
## 1 1e-03 0.21904762 0.05810478
## 2 1e-02 0.17142857 0.05701043
## 3 1e-01 0.08571429 0.05167880
## 4 1e+00 0.09285714 0.04821061
## 5 1e+01 0.09761905 0.05318053
## 6 1e+02 0.09523810 0.05019488
## 7 1e+03 0.09523810 0.05019488
```

```
tune_1$best.parameters
```

```
##   cost
## 3  0.1
```

```
bestSVMlinear=tune_1$best.model
summary(bestSVMlinear)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = trainset,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000)),
##     kernel = "linear", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  0.1
##
## Number of Support Vectors:  130
##
##  ( 64 66 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

- The best model is when cost is 0.01

Step 17: Predicting the response of the test set using the best model and calucate the mis-classification rate.

```
pred1=predict(bestSVMlinear, newdata=testset)
table(pred1, testset[,"Damage_High"])
```

```
##
## pred1   0   1
##     0 124  17
##     1   8  31
```

```
 tab1=table(pred1, testset[,"Damage_High"])
 misrate1=(tab1[1,2]+tab1[2,1])/sum(tab1)
 cat("Miscassification error is:", misrate1)
```

```
## Miscassification error is: 0.1388889
```

Step 18: Now tuning polynomial kernel with different values of degree and cost with the use of trainset. calculating the best model and predicting the mis classification error using test set just like the previous method.

```
set.seed(1)
 tune_2 = tune(svm, Damage_High ~ ., data = trainset, kernel = "polynomi
al", ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), d = c(2
:5)), scale=TRUE)
```

```
 summary(tune_2)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost d
##     1 3
##
## - best performance: 0.1333333
##
## - Detailed performance results:
##     cost d     error dispersion
## 1  1e-03 2 0.2190476 0.05810478
## 2  1e-02 2 0.2190476 0.05810478
## 3  1e-01 2 0.2166667 0.05662241
## 4  1e+00 2 0.1500000 0.05504299
## 5  1e+01 2 0.1428571 0.05939139
## 6  1e+02 2 0.1690476 0.06193528
## 7  1e+03 2 0.1809524 0.07291228
## 8  1e-03 3 0.2190476 0.05810478
## 9  1e-02 3 0.2190476 0.05810478
## 10 1e-01 3 0.2119048 0.05986674
## 11 1e+00 3 0.1333333 0.05044523
```

```
## 12 1e+01 3 0.1404762 0.06193528
## 13 1e+02 3 0.1809524 0.06753031
## 14 1e+03 3 0.2023810 0.06941609
## 15 1e-03 4 0.2190476 0.05810478
## 16 1e-02 4 0.2190476 0.05810478
## 17 1e-01 4 0.2142857 0.05723097
## 18 1e+00 4 0.1547619 0.05411977
## 19 1e+01 4 0.1523810 0.05167880
## 20 1e+02 4 0.1547619 0.05639949
## 21 1e+03 4 0.1809524 0.06369017
## 22 1e-03 5 0.2190476 0.05810478
## 23 1e-02 5 0.2190476 0.05810478
## 24 1e-01 5 0.2119048 0.05986674
## 25 1e+00 5 0.1666667 0.07009334
## 26 1e+01 5 0.1500000 0.05504299
## 27 1e+02 5 0.1571429 0.07462006
## 28 1e+03 5 0.1642857 0.06587780
```

```
tune_2$best.parameters
```

```
##    cost d
## 11    1 3
```

```
bestSVMPolynomial = tune_2$best.model
summary(bestSVMPolynomial)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = trainset,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), d = c
(2:5)),
##     kernel = "polynomial", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  polynomial
##        cost:  1
##      degree:  3
##      coef.0:  0
##
## Number of Support Vectors:  178
##
##  ( 78 100 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```
pred2=predict(bestSVMPolynomial, newdata=testset)
table(pred2, testset[,"Damage_High"])
```

```
##
## pred2   0   1
##     0 128  32
##     1   4  16
```

```
tab2=table(pred2, testset[,"Damage_High"])
misrate2=(tab2[1,2]+tab2[2,1])/sum(tab2)
cat("Miscassification error is:", misrate2)
```

```
## Miscassification error is: 0.2
```

- From the above output we have found out that the best model is when cost is 1 and degree is 3. And the misclassification error is found out as 0.2

Step 19: Now tuning radial kernel with different values of gamma with the use of trainset. calculating the best model and mis classification error.

```
set.seed(1)
tune_3 = tune(svm, Damage_High ~ ., data = trainset, kernel = "radial",
ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), gamma = c(0.
5, 1, 2, 3,4)), scale=TRUE)
```

```
summary(tune_3)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost gamma
##    10   0.5
##
## - best performance: 0.1952381
##
## - Detailed performance results:
##     cost gamma     error dispersion
## 1  1e-03   0.5 0.2190476 0.05810478
## 2  1e-02   0.5 0.2190476 0.05810478
## 3  1e-01   0.5 0.2190476 0.05810478
## 4  1e+00   0.5 0.2119048 0.05772412
## 5  1e+01   0.5 0.1952381 0.05240500
## 6  1e+02   0.5 0.1952381 0.05240500
## 7  1e+03   0.5 0.1952381 0.05240500
## 8  1e-03   1.0 0.2190476 0.05810478
## 9  1e-02   1.0 0.2190476 0.05810478
## 10 1e-01   1.0 0.2190476 0.05810478
## 11 1e+00   1.0 0.2190476 0.05810478
## 12 1e+01   1.0 0.2190476 0.06023386
## 13 1e+02   1.0 0.2190476 0.06023386
## 14 1e+03   1.0 0.2190476 0.06023386
## 15 1e-03   2.0 0.2190476 0.05810478
## 16 1e-02   2.0 0.2190476 0.05810478
```

```
## 17 1e-01   2.0 0.2190476 0.05810478
## 18 1e+00   2.0 0.2190476 0.05810478
## 19 1e+01   2.0 0.2190476 0.05810478
## 20 1e+02   2.0 0.2190476 0.05810478
## 21 1e+03   2.0 0.2190476 0.05810478
## 22 1e-03   3.0 0.2190476 0.05810478
## 23 1e-02   3.0 0.2190476 0.05810478
## 24 1e-01   3.0 0.2190476 0.05810478
## 25 1e+00   3.0 0.2190476 0.05810478
## 26 1e+01   3.0 0.2190476 0.05810478
## 27 1e+02   3.0 0.2190476 0.05810478
## 28 1e+03   3.0 0.2190476 0.05810478
## 29 1e-03   4.0 0.2190476 0.05810478
## 30 1e-02   4.0 0.2190476 0.05810478
## 31 1e-01   4.0 0.2190476 0.05810478
## 32 1e+00   4.0 0.2190476 0.05810478
## 33 1e+01   4.0 0.2190476 0.05810478
## 34 1e+02   4.0 0.2190476 0.05810478
## 35 1e+03   4.0 0.2190476 0.05810478
```

```r
tune_3$best.parameters
```

```
##   cost gamma
## 5   10   0.5
```

```r
bestSVMRadial = tune_3$best.model
summary(bestSVMRadial)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = trainset,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), gamma
= c(0.5,
##         1, 2, 3, 4)), kernel = "radial", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  10
##
## Number of Support Vectors:  395
##
##  ( 92 303 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```r
pred3=predict(bestSVMRadial, newdata=testset)
table(pred3, testset[,"Damage_High"])
```

```
##
## pred3   0   1
##     0 130  41
##     1   2   7
```

```
tab3=table(pred3, testset[,"Damage_High"])
misrate3=(tab3[1,2]+tab3[2,1])/sum(tab3)
cat("Miscassification error is:", misrate3)
```

```
## Miscassification error is: 0.2388889
```

- From the above output we have found out that the best model is when cost is 10 and gamma is 0.5. And the misclassification error is found out as 0.239

Step 20: Creating a data frame to show the miss classification error of different models to identify the best model

```
misClass <- data.frame(Model = c("SVM with Linear kernel",
                                 "SVM with Polynomial kernel",
                                 "SVM with Radial kernel"),
                       Mis.Rate = c(misrate1, misrate2, misrate3))
```

```
misClass
```

```
##                        Model  Mis.Rate
## 1     SVM with Linear kernel 0.1388889
## 2 SVM with Polynomial kernel 0.2000000
## 3     SVM with Radial kernel 0.2388889
```

- From the above data frame, it is clear that SVM with Linear kernel is the best model because Linear Kernel has less error comparing to other kernels.

2. Give the resultant accepted model based on your findings above. Justify your answer clearly. **(5 Marks)**

- According to the finding above Linear Kernel is found out to be the best model because Linear Kernal model has the less miss classification error. And also for the final confirmation I have checked confusion matrix for the misclassification and also for the response of actual and predicted values. Finally I have checked the accuracy of the linear model, which is calculated as 86%.

```
summary(bestSVMlinear)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = trainset,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000)),
```

```
##     kernel = "linear", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  0.1
##
## Number of Support Vectors:  130
##
##  ( 64 66 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

Confusion Matrix

```
Confusion_matrix <- table(pred1, testset[,"Damage_High"])
Confusion_matrix

##
## pred1   0   1
##     0 124  17
##     1   8  31

cat("misclassification rate :", misrate1)

## misclassification rate : 0.1388889
```

Model Accuracy

```
accuracy <- sum(diag(Confusion_matrix)) / sum(Confusion_matrix)

cat("Accuracy of the model is :", accuracy)

## Accuracy of the model is : 0.8611111
```

3. Apply an unsupervised learning technique of your choice to identify any interesting or hidden patterns in the dataset. Provide a clear explanation of the technique used and thoroughly describe your findings. **(10 Marks)**

K-Means Clustering and Hierarchical Clustering were applied as unsupervised learning techniques.

Steps Taken for Clustering Analysis

Data Preparation

Removed Irrelevant Variables:

- Columns Damage_High, Construction_Quality, and Insurance_Coverage were excluded as they are not useful for clustering.

```
bushfire_cluster <- bushfire %>% select(-Damage_High, -
Construction_Quality, -Insurance_Coverage)
```

- Dataset was scaled to standardize variables, ensuring all features contribute equally to clustering.

Checked Summary Statistics:

- Calculated mean and variance for all variables to understand data distribution and variability.

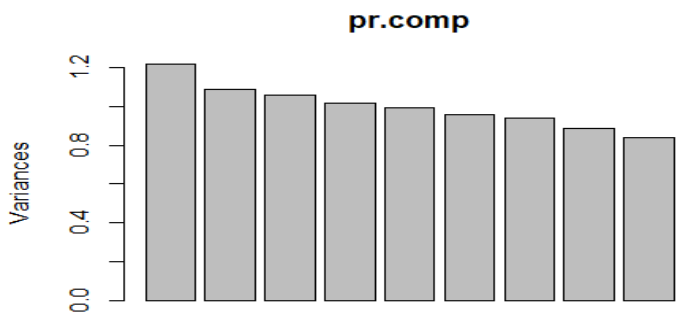
```
lapply(bushfire_cluster, mean)
sapply(bushfire_cluster, var)
```

Principal Component Analysis (PCA)

- PCA was applied to reduce the dataset's dimensionality and visualize patterns while preserving variance.

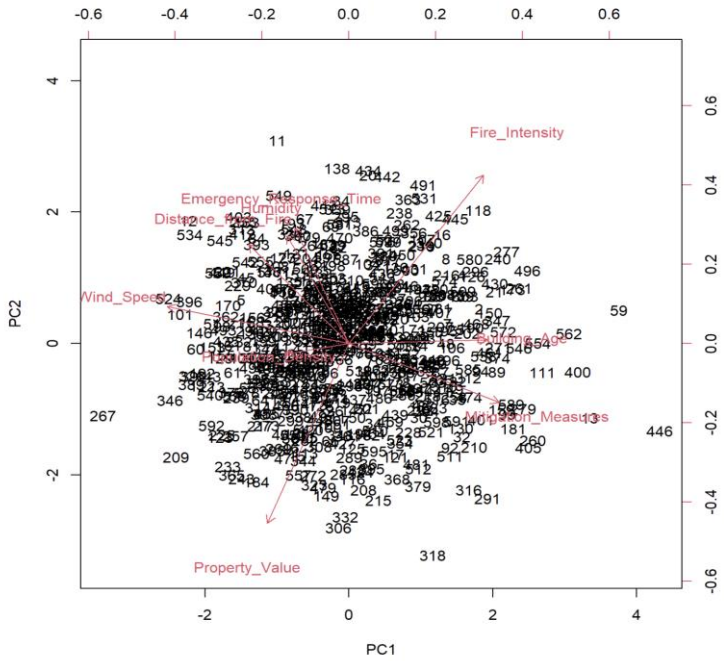
Findings:

- The first three principal components explained approximately 37.36% of the variance.
- A scree plot and cumulative variance plot were used to identify the most significant components.



Biplot:

- Visualized the contribution of variables to the principal components and potential groupings in the data.



K-Means Clustering

Performed K-Means with 2 and 3 Clusters:

Cluster 2 Results:

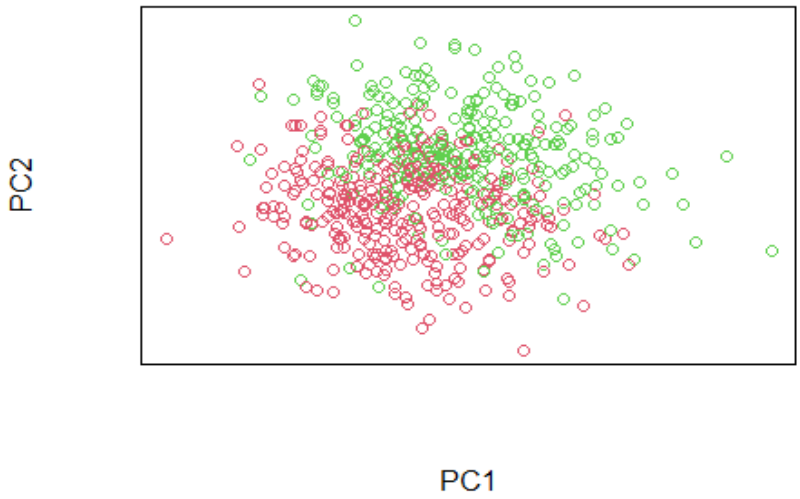
- Sizes: Cluster 1 (316 observations), Cluster 2 (284 observations).
- Cluster centers showed differences in Fire_Intensity, Humidity, and Mitigation_Measures.
- Within-cluster sum of squares (WSS): 75.2% of the variance was explained by the clustering.

```
kmeans_out

## K-means clustering with 2 clusters of sizes 316, 284
##
## Cluster means:
##   Fire_Intensity Distance_from_Fire Building_Age Property_Value
## 1       5435.066           7.003734     24.68354       7.496266
## 2      11889.442           7.049718     25.10563       7.035282
##   Population_Density Emergency_Response_Time Mitigation_Measures Wind
_Speed
## 1           597.6171                 14.92722            2.955696   24
.97627
## 2           598.9613                 15.01056            3.003521   25
.01338
##   Humidity
## 1 49.50889
## 2 50.06246
##
## Clustering vector:
```

```
##   [1] 1 2 1 2 2 1 2 2 2 1 2 1 2 2 1 2 1 1 1 2 2 2 2 2 2 2 2 2 1 1 2 2
2 2 1 1 2
##  [38] 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 2 1 2 1 1 2 2 1 2 1 1 1 2 1 2 2 2
1 2 2 2 1
##  [75] 1 1 1 2 1 1 1 2 1 2 1 1 2 2 2 1 1 2 1 2 1 1 2 1 1 1 2 1 1 2 1 2
2 2 1 1 2
## [112] 1 1 2 2 1 2 2 2 1 2 1 1 1 1 2 1 1 1 2 2 2 2 2 2 2 2 2 2 1 1 1 1
1 2 1 1 1
## [149] 1 2 2 1 1 1 1 1 2 1 1 1 1 1 2 1 1 2 2 1 1 1 2 1 2 2 2 2 1 2 2 2
2 1 2 1 2
## [186] 1 1 2 2 2 1 1 2 2 2 1 1 2 1 2 1 2 2 2 1 2 1 1 1 1 1 1 1 2 1 2 1
1 2 2 1 2
## [223] 2 2 1 1 1 2 2 2 2 1 2 1 1 1 1 2 1 2 2 2 1 2 1 2 1 2 2 2 1 1 2 1
2 2 1 1 1
## [260] 2 2 2 2 2 2 2 1 1 1 1 2 1 1 1 2 2 2 1 1 2 2 1 1 1 1 1 1 1 1 2 1
1 1 2 2 2
## [297] 2 1 1 2 2 1 2 2 2 1 1 1 1 1 1 1 2 1 1 2 2 1 1 2 2 1 1 2 1 1 2 1
1 2 2 1 2
## [334] 2 1 1 2 2 2 2 1 1 2 1 1 1 2 1 1 2 1 2 2 1 2 2 2 1 1 2 1 1 2 1 1
2 1 1 1 1
## [371] 1 1 2 1 2 2 2 1 1 2 2 2 1 2 2 2 1 1 1 1 2 1 2 2 1 1 1 1 1 2 2 1
2 2 1 1 2
## [408] 1 1 2 1 2 1 1 1 1 2 1 1 1 2 2 2 1 2 2 1 1 1 2 2 2 1 2 1 2 1 1 1
1 1 2 2 1
## [445] 2 2 2 1 1 2 2 1 1 1 1 2 2 2 1 1 2 1 1 1 2 1 1 2 1 2 1 2 1 2 2 1
2 1 1 2 1
## [482] 2 1 2 2 1 1 2 1 2 2 1 1 2 1 2 1 1 2 2 1 1 1 1 1 1 2 1 2 1 1 1 2
1 2 1 1 2
## [519] 1 2 1 1 1 1 2 2 2 1 2 1 2 2 1 2 1 2 1 2 2 1 2 1 2 1 2 2 1 2 2 1
1 1 2 2 2
## [556] 2 2 2 1 1 1 2 2 1 2 2 1 2 2 2 2 2 2 1 2 2 1 1 2 1 2 2 2 1 2 1 1
2 2 2 2 1
## [593] 2 1 1 2 1 1 2 1
##
## Within cluster sum of squares by cluster:
## [1] 1090005357  967451613
##  (between_SS / total_SS =  75.2 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.
withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

Visualized Clusters:

- Plotted clusters showing clearer separations with two clusters.



Cluster 3 Results:

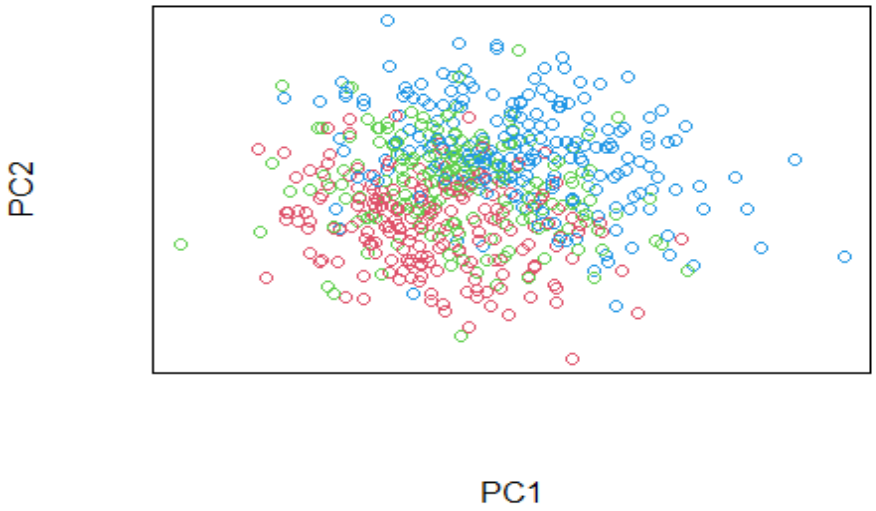
- Sizes: Cluster 1 (197), Cluster 2 (194), Cluster 3 (209).
- Cluster centers further distinguished groups, with higher separation between clusters.
- WSS: 88.4% of variance explained, indicating improved clustering.

```
## K-means clustering with 3 clusters of sizes 197, 194, 209
##
## Cluster means:
##   Fire_Intensity Distance_from_Fire Building_Age Property_Value
## 1       4242.686           7.065431     24.64975       7.491168
## 2       8232.687           7.036804     24.69588       7.426443
## 3      12732.690           6.977368     25.27751       6.939474
##   Population_Density Emergency_Response_Time Mitigation_Measures Wind
_Speed
## 1           596.3553                14.61421            3.005076   25
.23198
## 2           594.7784                15.48969            2.706186   24
.99845
## 3           603.2679                14.81340            3.205742   24
.76507
##   Humidity
## 1 47.97259
## 2 51.77041
## 3 49.61000
##
## Clustering vector:
##   [1] 1 3 2 3 3 1 2 3 2 2 3 2 3 2 1 3 1 1 2 3 3 3 2 3 3 3 2 2 1 1 3 3
3 3 1 2 3
##  [38] 1 1 1 1 2 2 2 1 1 1 2 1 3 1 2 3 1 2 1 1 3 3 2 3 1 2 1 3 2 3 3 3
2 3 2 3 1
##  [75] 2 1 2 2 2 1 1 3 2 3 1 2 3 3 3 1 1 3 2 3 1 1 3 1 2 2 2 2 2 3 2 3
3 2 2 1 3
```

```
## [112] 1 1 3 3 1 2 3 2 2 2 1 1 1 2 3 1 1 1 3 2 3 3 3 2 3 3 3 3 2 1 2 1
1 3 1 1 1
## [149] 1 3 3 2 2 1 1 2 2 1 2 1 2 2 2 2 2 2 3 1 2 1 2 1 3 3 3 2 2 2 3 2
3 1 3 1 2
## [186] 2 1 2 3 3 1 1 3 2 3 2 2 3 1 2 1 3 2 2 2 3 2 1 1 1 2 1 1 3 1 3 2
2 3 3 1 3
## [223] 3 3 1 2 2 2 3 3 2 2 2 1 1 2 1 3 1 3 2 3 1 2 1 3 2 3 3 3 1 1 2 1
2 3 1 2 2
## [260] 3 3 3 3 3 2 2 2 2 1 2 3 1 1 1 3 3 3 2 1 2 3 1 2 1 1 2 1 1 1 3 1
1 1 3 3 3
## [297] 3 1 1 3 3 1 3 3 2 2 1 1 2 2 2 2 3 1 2 3 3 1 2 3 3 1 1 3 1 1 3 1
2 3 2 1 3
## [334] 3 1 2 3 2 2 3 2 1 2 1 2 2 3 2 1 2 1 3 2 1 2 3 3 2 1 3 1 1 3 1 2
3 1 1 2 2
## [371] 2 2 3 2 2 3 3 1 1 3 2 3 2 3 2 3 1 1 1 1 3 1 3 3 1 1 1 1 1 3 3 1
3 2 2 2 3
## [408] 1 2 3 1 3 1 2 1 1 3 1 2 1 2 2 2 1 3 2 1 2 1 3 3 2 2 3 1 2 1 2 1
1 1 2 3 1
## [445] 3 3 3 1 2 3 2 2 1 1 1 3 3 3 1 1 3 2 2 1 2 1 1 3 1 3 1 3 2 3 3 2
2 1 1 3 1
## [482] 3 1 3 3 1 1 3 2 3 3 1 2 3 1 3 1 1 3 3 2 2 1 1 2 1 2 2 3 2 2 1 3
2 3 1 2 3
## [519] 1 3 1 2 1 1 2 3 3 1 3 1 3 3 2 3 1 3 2 3 2 1 3 1 3 1 3 3 1 3 3 2
2 1 2 3 2
## [556] 2 3 2 2 1 1 3 3 1 2 3 2 3 2 3 2 3 2 2 3 3 2 1 2 2 3 3 3 2 3 1 1
3 3 2 2 1
## [593] 3 1 1 3 2 2 3 1
##
## Within cluster sum of squares by cluster:
## [1] 290622002 286399109 382747287
##  (between_SS / total_SS =  88.4 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.
withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

Visualized Clusters:

- Plotted clusters showing clearer separations with three clusters.



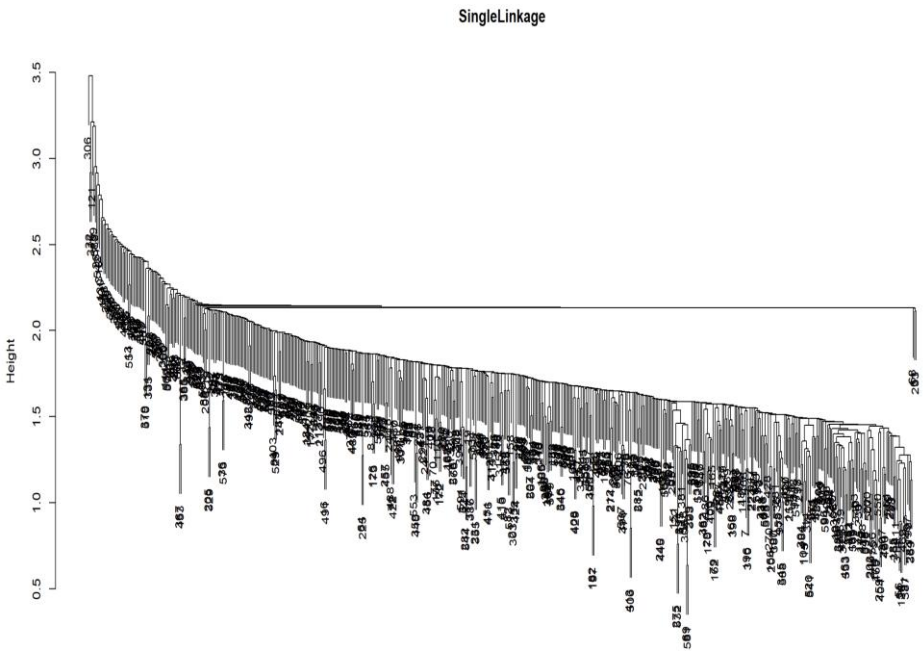
Hierarchical Clustering

Linkage Methods Tested:

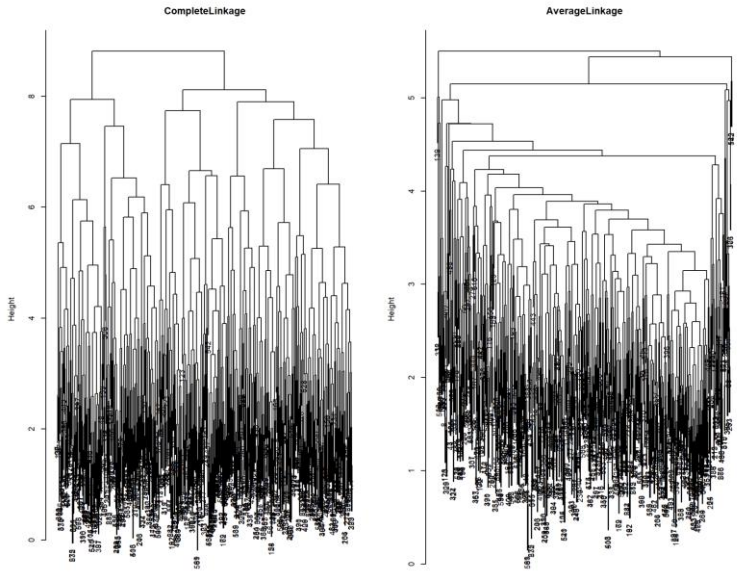
Complete, **Average**, and **Single** linkage methods were applied.

- Hierarchical dendrograms visualized group hierarchies for better interpretability.

Single :



Complete and Average:



Findings:

- Complete linkage provided the best separation of clusters.
- Tested with cutree() to derive clusters and validated against K-Means clusters.

Findings

K-Means Insights:

2 Clusters:

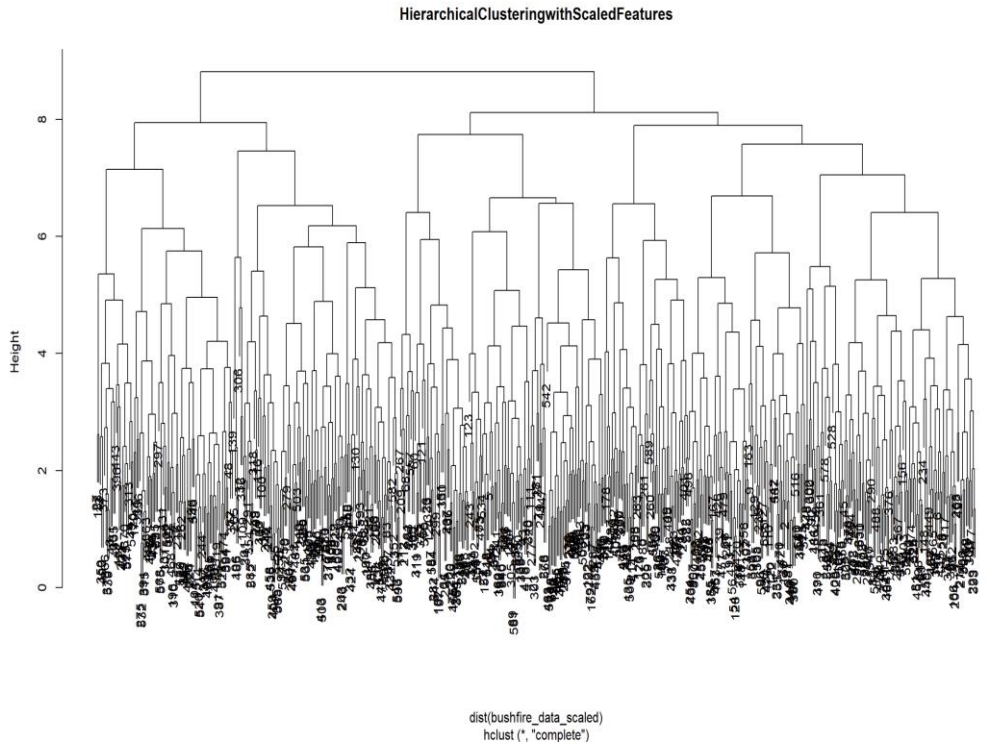
- Cluster 1: Lower Fire_Intensity, slightly lower Humidity, and fewer Mitigation_Measures.
- Cluster 2: Higher Fire_Intensity and Humidity.

3 Clusters:

- Cluster 1: Very low Fire_Intensity, highest Humidity.
- Cluster 2: Moderate Fire_Intensity, moderate Humidity.
- Cluster 3: Highest Fire_Intensity, lowest Humidity.

Hierarchical Clustering Insights:

- Provided a hierarchical view of group relationships.
- Supported findings from K-Means, showing distinct patterns for Fire_Intensity, Mitigation_Measures, and Humidity.



Conclusions

Best Model:

- **K-Means with 3 Clusters** is recommended as it explained the highest variance (88.4%) and provided interpretable clusters.
- Hierarchical clustering validated the separation of these clusters.

Key Observations:

- Clusters reveal insights about property risk based on Fire_Intensity, Distance_from_Fire, and mitigation strategies.
- Distinct patterns highlight groups with high fire risk, moderate risk, and low risk.

Potential Applications:

- Use clustering results to target specific properties for improved fire mitigation measures.

4. What are your conclusion and recommendations for this problem? [Hint: Use your results and findings from previous questions to answer this question] **(6 Marks)**

Supervised Learning Insights

- The SVM with Linear Kernel was determined to be the best model for classifying damage claims based on features like fire intensity, property characteristics, and mitigation measures.
- Accuracy: 86%
- Misclassification Rate: 0.1388
- Optimal Parameters: Cost parameter (C) = 0.1, chosen through 10-fold cross-validation.

2. Clustering Findings (Unsupervised Learning)

K-Means Clustering:

Two Clusters:

- Cluster 1: Lower fire intensity, moderate humidity, and fewer mitigation measures.
- Cluster 2: Higher fire intensity and advanced mitigation measures.

Three Clusters:

- Cluster 1: Very low fire intensity and high humidity.
- Cluster 2: Moderate fire intensity and moderate humidity.
- Cluster 3: High fire intensity and low humidity, indicating higher-risk properties.

Hierarchical Clustering:

- Supported the findings of K-Means clustering, providing consistent patterns in property risk levels based on fire intensity and mitigation efforts.

Combined Insights

- The clustering analysis complements the SVM model by identifying risk-based groupings that align with the classification results, confirming the effectiveness of the features used.

Recommendations

Implementation of the Linear Kernel Model

- Use the SVM with Linear Kernel for real-time classification of properties likely to incur high damage claims.
- Automate this process within the company's risk assessment pipeline to prioritize high-risk properties for additional support.

Leverage Clustering for Risk Segmentation

- Utilize clustering insights to segment properties into risk categories:
- Low Risk: Properties with low fire intensity and adequate mitigation.

- Moderate Risk: Properties with moderate fire intensity and average mitigation measures.
- High Risk: Properties with high fire intensity and insufficient mitigation.

Enhance Mitigation Strategies

➢ Focus on high-risk clusters identified in clustering analysis to:

- Improve mitigation measures for properties with low readiness.
- Offer incentives for adopting fire-resistant construction and better insurance coverage.

Monitor and Update Models Regularly

- Continuously update the SVM model and clustering process with new data to ensure ongoing accuracy and relevance.
- Evaluate model performance periodically using misclassification rates and clustering consistency.

Further Analysis and Tools

- Consider exploring ensemble methods or advanced unsupervised techniques like Db Scan to improve insights.
- Use these findings to guide policy changes, resource allocation, and premium adjustments for properties at varying risk levels.

--- End of questions ---

APPENDIX

```
# Load necessary libraries
library(e1071)
library(caret)

library(dplyr)
```

QUESTION 1 Data Preprocessing

```
# Load the dataset
bushfire_data <- read.csv("BushFireData.csv")

#Structure of the data
str(bushfire_data)

## 'data.frame':    600 obs. of  13 variables:
##  $ ClaimID                : chr  "DQTA26862" "RDAQ32534" "WTLA41817" "Y
HZL16317" ...
##  $ Fire_Intensity         : num  5738 12248 7317 13479 14226 ...
##  $ Distance_from_Fire     : num  5.57 5.49 5.12 4.89 6.13 ...
##  $ Building_Age           : int  19 27 16 30 21 27 24 24 36 30 ...
##  $ Property_Value         : num  9.76 5.89 3.98 4.91 9.31 ...
##  $ Population_Density     : int  520 433 634 576 644 476 500 591 487 44
```

```
1 ...
##  $ Emergency_Response_Time: int  16 18 17 14 12 10 16 19 15 18 ...
##  $ Mitigation_Measures    : int  3 3 4 2 2 1 5 7 2 3 ...
##  $ Construction_Quality   : chr  "Good" "Good" "Good" "Good" ...
##  $ Insurance_Coverage     : chr  "Fully" "None" "Partially" "Partially"
...
##  $ Wind_Speed             : num  24.9 21 25.3 22.3 37.2 31.8 27.8 22.5
32.2 20.6 ...
##  $ Humidity               : num  35.3 28.2 44.5 53.8 73.8 ...
##  $ Damage_Claims          : num  7.02 7.45 4.3 7.78 9.46 4.54 7.14 7.73
6.03 5.37 ...
```

```
# Convert categorical variables to factors
bushfire_data$Construction_Quality <- as.factor(bushfire_data$Construction
_Quality)

bushfire_data$Insurance_Coverage <- as.factor(bushfire_data$Insurance_Cove
rage)

#binary response variable
bushfire_data$Damage_High <- ifelse(bushfire_data$Damage_Claims > 7.5, 1,
0)
bushfire_data$Damage_High <- as.factor(bushfire_data$Damage_High)

str(bushfire_data)
```

```
## 'data.frame':    600 obs. of  14 variables:
##  $ ClaimID                : chr  "DQTA26862" "RDAQ32534" "WTLA41817" "Y
HZL16317" ...
##  $ Fire_Intensity         : num  5738 12248 7317 13479 14226 ...
##  $ Distance_from_Fire     : num  5.57 5.49 5.12 4.89 6.13 ...
##  $ Building_Age           : int  19 27 16 30 21 27 24 24 36 30 ...
##  $ Property_Value         : num  9.76 5.89 3.98 4.91 9.31 ...
##  $ Population_Density     : int  520 433 634 576 644 476 500 591 487 44
1 ...
##  $ Emergency_Response_Time: int  16 18 17 14 12 10 16 19 15 18 ...
##  $ Mitigation_Measures    : int  3 3 4 2 2 1 5 7 2 3 ...
##  $ Construction_Quality   : Factor w/ 2 levels "Bad","Good": 2 2 2 2 2
2 2 2 2 2 ...
##  $ Insurance_Coverage     : Factor w/ 3 levels "Fully","None",..: 1 2 3
3 2 1 3 1 1 3 ...
##  $ Wind_Speed             : num  24.9 21 25.3 22.3 37.2 31.8 27.8 22.5
32.2 20.6 ...
##  $ Humidity               : num  35.3 28.2 44.5 53.8 73.8 ...
##  $ Damage_Claims          : num  7.02 7.45 4.3 7.78 9.46 4.54 7.14 7.73
6.03 5.37 ...
##  $ Damage_High            : Factor w/ 2 levels "0","1": 1 1 1 2 2 1 1 2
1 1 ...
```

```
head(bushfire_data$Damage_High)
```

```
## [1] 0 0 0 1 1 0
## Levels: 0 1
```

```r
# Remove irrelevent columns
bushfire <- bushfire_data %>% select(-ClaimID, -Damage_Claims)

str(bushfire)
```

```
## 'data.frame':    600 obs. of  12 variables:
##  $ Fire_Intensity         : num  5738 12248 7317 13479 14226 ...
##  $ Distance_from_Fire     : num  5.57 5.49 5.12 4.89 6.13 ...
##  $ Building_Age           : int  19 27 16 30 21 27 24 24 36 30 ...
##  $ Property_Value         : num  9.76 5.89 3.98 4.91 9.31 ...
##  $ Population_Density     : int  520 433 634 576 644 476 500 591 487 44
1 ...
##  $ Emergency_Response_Time: int  16 18 17 14 12 10 16 19 15 18 ...
##  $ Mitigation_Measures    : int  3 3 4 2 2 1 5 7 2 3 ...
##  $ Construction_Quality   : Factor w/ 2 levels "Bad","Good": 2 2 2 2 2
2 2 2 2 2 ...
##  $ Insurance_Coverage     : Factor w/ 3 levels "Fully","None",..: 1 2 3
3 2 1 3 1 1 3 ...
##  $ Wind_Speed             : num  24.9 21 25.3 22.3 37.2 31.8 27.8 22.5
32.2 20.6 ...
##  $ Humidity               : num  35.3 28.2 44.5 53.8 73.8 ...
##  $ Damage_High            : Factor w/ 2 levels "0","1": 1 1 1 2 2 1 1 2
1 1 ...
```

Linear Kernal

```r
svm_fit1=svm(Damage_High ~.,data = bushfire, kernel="linear", cost=1,scale
=TRUE)

 summary(svm_fit1)
```

```
##
## Call:
## svm(formula = Damage_High ~ ., data = bushfire, kernel = "linear",
##     cost = 1, scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  1
##
## Number of Support Vectors:  159
##
##  ( 80 79 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```r
 set.seed(1)
tune_out = tune(svm, Damage_High ~ ., data = bushfire, kernel = "linear",
ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000)),scale=TRUE)
```

```r
summary(tune_out)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost
##   0.1
##
## - best performance: 0.1016667
##
## - Detailed performance results:
##    cost     error dispersion
## 1 1e-03 0.2333333 0.04230985
## 2 1e-02 0.1583333 0.04795188
## 3 1e-01 0.1016667 0.04743416
## 4 1e+00 0.1183333 0.05238155
## 5 1e+01 0.1200000 0.04766136
## 6 1e+02 0.1200000 0.04766136
## 7 1e+03 0.1200000 0.04766136
```

```r
tune_out$best.parameters
```

```
##   cost
## 3  0.1
```

```r
best_model1 <- tune_out$best.model
summary(best_model1)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = bushfire,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000)),
##     kernel = "linear", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  0.1
##
## Number of Support Vectors:  194
##
##  ( 99 95 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

Polynomial kernel

```r
set.seed(1)
tune_out2 = tune(svm, Damage_High ~ ., data = bushfire, kernel = "polynomi
al",
ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), d = c(2:5)),
scale=TRUE)

summary(tune_out2)

##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost d
##    10 2
##
## - best performance: 0.125
##
## - Detailed performance results:
##     cost d     error dispersion
## 1  1e-03 2 0.2333333 0.04230985
## 2  1e-02 2 0.2333333 0.04230985
## 3  1e-01 2 0.2233333 0.04021547
## 4  1e+00 2 0.1533333 0.03832931
## 5  1e+01 2 0.1250000 0.03068156
## 6  1e+02 2 0.1666667 0.03928371
## 7  1e+03 2 0.1766667 0.03784308
## 8  1e-03 3 0.2333333 0.04230985
## 9  1e-02 3 0.2333333 0.04230985
## 10 1e-01 3 0.2183333 0.04406084
## 11 1e+00 3 0.1550000 0.03853009
## 12 1e+01 3 0.1583333 0.03356402
## 13 1e+02 3 0.1683333 0.04116363
## 14 1e+03 3 0.2000000 0.03600411
## 15 1e-03 4 0.2333333 0.04230985
## 16 1e-02 4 0.2333333 0.04230985
## 17 1e-01 4 0.2200000 0.04430534
## 18 1e+00 4 0.1716667 0.04377975
## 19 1e+01 4 0.1600000 0.03944053
## 20 1e+02 4 0.1766667 0.05676462
## 21 1e+03 4 0.1850000 0.05178898
## 22 1e-03 5 0.2333333 0.04230985
## 23 1e-02 5 0.2333333 0.04230985
## 24 1e-01 5 0.2200000 0.04430534
## 25 1e+00 5 0.1850000 0.04996913
## 26 1e+01 5 0.1600000 0.03617516
## 27 1e+02 5 0.1666667 0.05030770
## 28 1e+03 5 0.1750000 0.04461772

tune_out2$best.parameters

##   cost d
## 5   10 2
```

```r
best_model2 = tune_out2$best.model
summary(best_model2)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = bushfire,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), d = c(2
:5)),
##     kernel = "polynomial", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  polynomial
##        cost:  10
##      degree:  2
##      coef.0:  0
##
## Number of Support Vectors:  187
##
##  ( 103 84 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```r
#radial kernel
set.seed(1)
tune_out3 = tune(svm, Damage_High ~ ., data = bushfire, kernel = "radial",
ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), gamma = c(0.5,
1, 2, 3,4)), scale=TRUE)

summary(tune_out3)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost gamma
##    10   0.5
##
## - best performance: 0.2
##
## - Detailed performance results:
##     cost gamma     error dispersion
## 1  1e-03   0.5 0.2333333 0.04230985
## 2  1e-02   0.5 0.2333333 0.04230985
## 3  1e-01   0.5 0.2333333 0.04230985
## 4  1e+00   0.5 0.2200000 0.04567734
## 5  1e+01   0.5 0.2000000 0.05270463
## 6  1e+02   0.5 0.2000000 0.05270463
```

```
## 7  1e+03   0.5 0.2000000 0.05270463
## 8  1e-03   1.0 0.2333333 0.04230985
## 9  1e-02   1.0 0.2333333 0.04230985
## 10 1e-01   1.0 0.2333333 0.04230985
## 11 1e+00   1.0 0.2333333 0.04230985
## 12 1e+01   1.0 0.2350000 0.04335470
## 13 1e+02   1.0 0.2350000 0.04335470
## 14 1e+03   1.0 0.2350000 0.04335470
## 15 1e-03   2.0 0.2333333 0.04230985
## 16 1e-02   2.0 0.2333333 0.04230985
## 17 1e-01   2.0 0.2333333 0.04230985
## 18 1e+00   2.0 0.2333333 0.04230985
## 19 1e+01   2.0 0.2333333 0.04230985
## 20 1e+02   2.0 0.2333333 0.04230985
## 21 1e+03   2.0 0.2333333 0.04230985
## 22 1e-03   3.0 0.2333333 0.04230985
## 23 1e-02   3.0 0.2333333 0.04230985
## 24 1e-01   3.0 0.2333333 0.04230985
## 25 1e+00   3.0 0.2333333 0.04230985
## 26 1e+01   3.0 0.2333333 0.04230985
## 27 1e+02   3.0 0.2333333 0.04230985
## 28 1e+03   3.0 0.2333333 0.04230985
## 29 1e-03   4.0 0.2333333 0.04230985
## 30 1e-02   4.0 0.2333333 0.04230985
## 31 1e-01   4.0 0.2333333 0.04230985
## 32 1e+00   4.0 0.2333333 0.04230985
## 33 1e+01   4.0 0.2333333 0.04230985
## 34 1e+02   4.0 0.2333333 0.04230985
## 35 1e+03   4.0 0.2333333 0.04230985
```

```
tune_out3$best.parameters
```

```
##   cost gamma
## 5   10   0.5
```

```
best_model3 = tune_out3$best.model
summary(best_model3)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = bushfire,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), gamma =
c(0.5,
##         1, 2, 3, 4)), kernel = "radial", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  10
##
## Number of Support Vectors:  544
##
##  ( 405 139 )
##
```

```
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```r
#Train/Test division
set.seed(1)
trid <- sample(1:nrow(bushfire), nrow(bushfire)*0.7)
trainset <- bushfire[trid,]
testset <- bushfire[-trid,]
```

```r
set.seed(1)
tune_1=tune(svm, Damage_High ~., data = trainset, kernel="linear", scale =
TRUE, ranges = list(cost=c(0.001, 0.01, 0.1, 1, 10, 100, 1000)))
```

```r
summary(tune_1)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost
##   0.1
##
## - best performance: 0.08571429
##
## - Detailed performance results:
##    cost      error dispersion
## 1 1e-03 0.21904762 0.05810478
## 2 1e-02 0.17142857 0.05701043
## 3 1e-01 0.08571429 0.05167880
## 4 1e+00 0.09285714 0.04821061
## 5 1e+01 0.09761905 0.05318053
## 6 1e+02 0.09523810 0.05019488
## 7 1e+03 0.09523810 0.05019488
```

```r
tune_1$best.parameters
```

```
##   cost
## 3  0.1
```

```r
bestSVMlinear=tune_1$best.model
summary(bestSVMlinear)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = trainset,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000)),
##     kernel = "linear", scale = TRUE)
##
##
## Parameters:
```

```
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  0.1
##
## Number of Support Vectors:  130
##
##  ( 64 66 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```
pred1=predict(bestSVMlinear, newdata=testset)
table(pred1, testset[,"Damage_High"])
```

```
##
## pred1   0   1
##     0 124  17
##     1   8  31
```

```
 tab1=table(pred1, testset[,"Damage_High"])
 misrate1=(tab1[1,2]+tab1[2,1])/sum(tab1)
 cat("Miscassification error is:", misrate1)
```

```
## Miscassification error is: 0.1388889
```

```
#Polynomial basis kernel
set.seed(1)
tune_2 = tune(svm, Damage_High ~ ., data = trainset, kernel = "polynomial"
, ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), d = c(2:5)),
scale=TRUE)
```

```
 summary(tune_2)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost d
##     1 3
##
## - best performance: 0.1333333
##
## - Detailed performance results:
##     cost d     error dispersion
## 1  1e-03 2 0.2190476 0.05810478
## 2  1e-02 2 0.2190476 0.05810478
## 3  1e-01 2 0.2166667 0.05662241
## 4  1e+00 2 0.1500000 0.05504299
## 5  1e+01 2 0.1428571 0.05939139
## 6  1e+02 2 0.1690476 0.06193528
```

```
## 7  1e+03 2 0.1809524 0.07291228
## 8  1e-03 3 0.2190476 0.05810478
## 9  1e-02 3 0.2190476 0.05810478
## 10 1e-01 3 0.2119048 0.05986674
## 11 1e+00 3 0.1333333 0.05044523
## 12 1e+01 3 0.1404762 0.06193528
## 13 1e+02 3 0.1809524 0.06753031
## 14 1e+03 3 0.2023810 0.06941609
## 15 1e-03 4 0.2190476 0.05810478
## 16 1e-02 4 0.2190476 0.05810478
## 17 1e-01 4 0.2142857 0.05723097
## 18 1e+00 4 0.1547619 0.05411977
## 19 1e+01 4 0.1523810 0.05167880
## 20 1e+02 4 0.1547619 0.05639949
## 21 1e+03 4 0.1809524 0.06369017
## 22 1e-03 5 0.2190476 0.05810478
## 23 1e-02 5 0.2190476 0.05810478
## 24 1e-01 5 0.2119048 0.05986674
## 25 1e+00 5 0.1666667 0.07009334
## 26 1e+01 5 0.1500000 0.05504299
## 27 1e+02 5 0.1571429 0.07462006
## 28 1e+03 5 0.1642857 0.06587780
```

```
tune_2$best.parameters
```

```
##    cost d
## 11    1 3
```

```
bestSVMPolynomial = tune_2$best.model
summary(bestSVMPolynomial)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = trainset,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), d = c(2
:5)),
##     kernel = "polynomial", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  polynomial
##        cost:  1
##      degree:  3
##      coef.0:  0
##
## Number of Support Vectors:  178
##
##  ( 78 100 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```r
pred2=predict(bestSVMPolynomial, newdata=testset)
table(pred2, testset[,"Damage_High"])
```

```
##
## pred2   0   1
##     0 128  32
##     1   4  16
```

```r
tab2=table(pred2, testset[,"Damage_High"])
misrate2=(tab2[1,2]+tab2[2,1])/sum(tab2)
cat("Miscassification error is:", misrate2)
```

```
## Miscassification error is: 0.2
```

```r
#Radial basis kernel:
set.seed(1)
tune_3 = tune(svm, Damage_High ~ ., data = trainset, kernel = "radial", ra
nges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), gamma = c(0.5, 1
, 2, 3,4)), scale=TRUE)
```

```r
summary(tune_3)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost gamma
##    10   0.5
##
## - best performance: 0.1952381
##
## - Detailed performance results:
##     cost gamma     error dispersion
## 1  1e-03   0.5 0.2190476 0.05810478
## 2  1e-02   0.5 0.2190476 0.05810478
## 3  1e-01   0.5 0.2190476 0.05810478
## 4  1e+00   0.5 0.2119048 0.05772412
## 5  1e+01   0.5 0.1952381 0.05240500
## 6  1e+02   0.5 0.1952381 0.05240500
## 7  1e+03   0.5 0.1952381 0.05240500
## 8  1e-03   1.0 0.2190476 0.05810478
## 9  1e-02   1.0 0.2190476 0.05810478
## 10 1e-01   1.0 0.2190476 0.05810478
## 11 1e+00   1.0 0.2190476 0.05810478
## 12 1e+01   1.0 0.2190476 0.06023386
## 13 1e+02   1.0 0.2190476 0.06023386
## 14 1e+03   1.0 0.2190476 0.06023386
## 15 1e-03   2.0 0.2190476 0.05810478
## 16 1e-02   2.0 0.2190476 0.05810478
## 17 1e-01   2.0 0.2190476 0.05810478
## 18 1e+00   2.0 0.2190476 0.05810478
## 19 1e+01   2.0 0.2190476 0.05810478
## 20 1e+02   2.0 0.2190476 0.05810478
```

```
## 21 1e+03   2.0 0.2190476 0.05810478
## 22 1e-03   3.0 0.2190476 0.05810478
## 23 1e-02   3.0 0.2190476 0.05810478
## 24 1e-01   3.0 0.2190476 0.05810478
## 25 1e+00   3.0 0.2190476 0.05810478
## 26 1e+01   3.0 0.2190476 0.05810478
## 27 1e+02   3.0 0.2190476 0.05810478
## 28 1e+03   3.0 0.2190476 0.05810478
## 29 1e-03   4.0 0.2190476 0.05810478
## 30 1e-02   4.0 0.2190476 0.05810478
## 31 1e-01   4.0 0.2190476 0.05810478
## 32 1e+00   4.0 0.2190476 0.05810478
## 33 1e+01   4.0 0.2190476 0.05810478
## 34 1e+02   4.0 0.2190476 0.05810478
## 35 1e+03   4.0 0.2190476 0.05810478
```

```
tune_3$best.parameters
```

```
##   cost gamma
## 5   10   0.5
```

```
bestSVMRadial = tune_3$best.model
summary(bestSVMRadial)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = trainset,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000), gamma =
c(0.5,
##         1, 2, 3, 4)), kernel = "radial", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  10
##
## Number of Support Vectors:  395
##
##  ( 92 303 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```
pred3=predict(bestSVMRadial, newdata=testset)
table(pred3, testset[,"Damage_High"])
```

```
##
## pred3   0   1
##     0 130  41
##     1   2   7
```

```r
tab3=table(pred3, testset[,"Damage_High"])
misrate3=(tab3[1,2]+tab3[2,1])/sum(tab3)
cat("Miscassification error is:", misrate3)
```

```
## Miscassification error is: 0.2388889
```

```r
#Misclassification errors of each model
misClass <- data.frame(Model = c("SVM with Linear kernel",
                                 "SVM with Polynomial kernel",
                                 "SVM with Radial kernel"),
                       Mis.Rate = c(misrate1, misrate2, misrate3))

misClass
```

```
##                        Model  Mis.Rate
## 1     SVM with Linear kernel 0.1388889
## 2 SVM with Polynomial kernel 0.2000000
## 3     SVM with Radial kernel 0.2388889
```

QUESTION 2

```r
summary(bestSVMlinear)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = Damage_High ~ ., data = trainset,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 10, 100, 1000)),
##     kernel = "linear", scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  0.1
##
## Number of Support Vectors:  130
##
##  ( 64 66 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

```r
Confusion_matrix <- table(pred1, testset[,"Damage_High"])
Confusion_matrix
```

```
##
## pred1   0   1
##     0 124  17
##     1   8  31
```

```r
cat("misclassification rate :", misrate1)
```

```
## misclassification rate : 0.1388889
```

```r
accuracy <- sum(diag(Confusion_matrix)) / sum(Confusion_matrix)

cat("Accuracy of the model is :", accuracy)

## Accuracy of the model is : 0.8611111
```

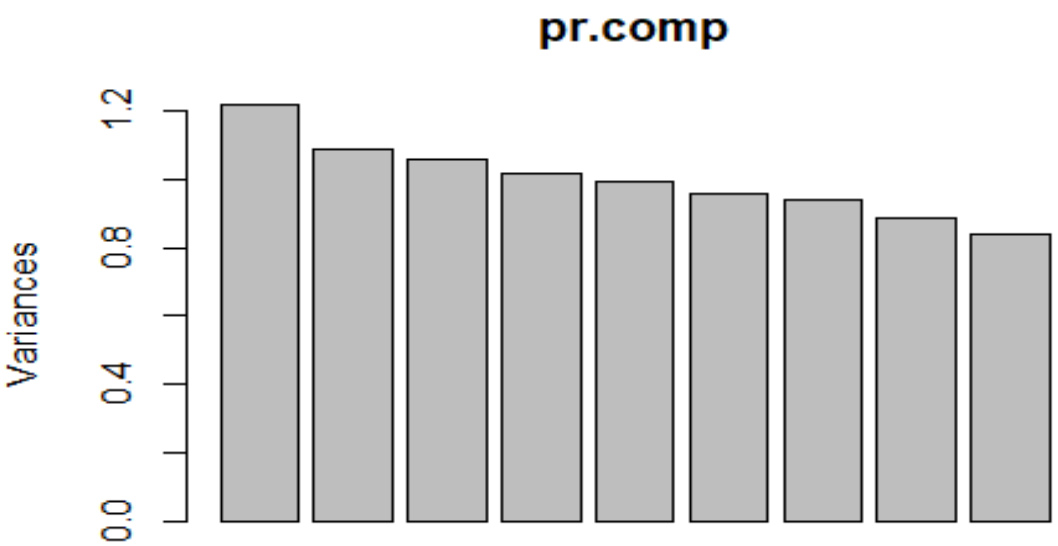
QUESTION 3

```r
#clear the irrelevent variable
bushfire_cluster <- bushfire %>% select(-Damage_High, -Construction_Qualit
y, -Insurance_Coverage)
str(bushfire_cluster)

## 'data.frame':    600 obs. of  9 variables:
##  $ Fire_Intensity         : num  5738 12248 7317 13479 14226 ...
##  $ Distance_from_Fire     : num  5.57 5.49 5.12 4.89 6.13 ...
##  $ Building_Age           : int  19 27 16 30 21 27 24 24 36 30 ...
##  $ Property_Value         : num  9.76 5.89 3.98 4.91 9.31 ...
##  $ Population_Density     : int  520 433 634 576 644 476 500 591 487 44
1 ...
##  $ Emergency_Response_Time: int  16 18 17 14 12 10 16 19 15 18 ...
##  $ Mitigation_Measures    : int  3 3 4 2 2 1 5 7 2 3 ...
##  $ Wind_Speed             : num  24.9 21 25.3 22.3 37.2 31.8 27.8 22.5
32.2 20.6 ...
##  $ Humidity               : num  35.3 28.2 44.5 53.8 73.8 ...

lapply(bushfire_cluster, mean)

## $Fire_Intensity
## [1] 8490.138
##
## $Distance_from_Fire
## [1] 7.0255
##
## $Building_Age
## [1] 24.88333
##
## $Property_Value
## [1] 7.278067
##
## $Population_Density
## [1] 598.2533
##
## $Emergency_Response_Time
## [1] 14.96667
##
## $Mitigation_Measures
## [1] 2.978333
##
## $Wind_Speed
## [1] 24.99383
##
## $Humidity
## [1] 49.77092
```

```r
sapply(bushfire_cluster, var)
```

```
##          Fire_Intensity      Distance_from_Fire            Building_Age
##            1.383053e+07            4.085310e+00            2.476099e+01
##          Property_Value      Population_Density Emergency_Response_Time
##            4.729333e+00            6.317351e+03            1.607568e+01
##     Mitigation_Measures              Wind_Speed                Humidity
##            3.039597e+00            2.689868e+01            3.507614e+02
```

```r
pr.comp <- prcomp(bushfire_cluster, scale. = T)

screeplot(pr.comp)
```




```r
pr.comp$sdev
```

```
## [1] 1.1032197 1.0419733 1.0295406 1.0070118 0.9977733 0.9800100 0.96955
26
## [8] 0.9421595 0.9162443
```

```r
pr.comp$center
```

```
##          Fire_Intensity      Distance_from_Fire            Building_Age
##             8490.137667                7.025500               24.883333
##          Property_Value      Population_Density Emergency_Response_Time
##                7.278067              598.253333               14.966667
##     Mitigation_Measures              Wind_Speed                Humidity
##                2.978333               24.993833               49.770917
```

```r
pr.comp$rotation
```

```
##                                PC1         PC2          PC3        PC4
## Fire_Intensity           0.3879646  0.53114764  0.277551664 -0.1241035
```

```
## Distance_from_Fire      -0.2886636  0.31299597  0.283544133  0.1461189
## Building_Age             0.3992121  0.01092616  0.188956016 -0.2397889
## Property_Value          -0.2343575 -0.56694049  0.046357169  0.1135330
## Population_Density      -0.1844550 -0.03471533  0.665073999 -0.2027555
## Emergency_Response_Time -0.1559268  0.36447621 -0.481160270 -0.1691881
## Mitigation_Measures      0.4326790 -0.18691331  0.233395512  0.5172214
## Wind_Speed              -0.5238593  0.11616106  0.276349250 -0.1083837
## Humidity                -0.1785676  0.34043615 -0.005782125  0.7374684
##                                PC5         PC6         PC7         PC8
## Fire_Intensity           0.06774495  0.05978999  0.17105479 -0.46566106
## Distance_from_Fire       0.39809326 -0.39041991 -0.63245462 -0.04322718
## Building_Age             0.63950630  0.32255633  0.06747088  0.33870550
## Property_Value           0.55375337 -0.06649410  0.24063293 -0.27686720
## Population_Density      -0.19223228 -0.34593910  0.37214303  0.42027466
## Emergency_Response_Time  0.26021220 -0.48561646  0.48327720 -0.01756541
## Mitigation_Measures     -0.06798850 -0.34429795  0.17163800 -0.30500609
## Wind_Speed              -0.03133774  0.41519685  0.19695159 -0.45038549
## Humidity                 0.10513399  0.29850745  0.26074521  0.34219758
##                                PC9
## Fire_Intensity          -0.46974977
## Distance_from_Fire       0.06469508
## Building_Age             0.33914973
## Property_Value          -0.40372316
## Population_Density      -0.09792678
## Emergency_Response_Time  0.21284964
## Mitigation_Measures      0.45848552
## Wind_Speed               0.45711549
## Humidity                -0.15190984
```

```r
head(pr.comp$x)
```

```
##             PC1        PC2        PC3          PC4         PC5        P
C6
## [1,] -0.5245971 -1.4173303 -1.3564853 -0.004936077 -0.28599453 -0.24702
97
## [2,]  1.8103159  0.5301982 -1.8342258 -0.875235087  0.18356445  0.22134
66
## [3,] -0.0967987  0.3485307 -0.5527159  0.069854381 -2.40385667 -0.78875
07
## [4,]  1.5715451  1.0495303 -0.1293591 -0.669977650 -0.20716960  1.15200
80
## [5,] -1.4997559  0.6674983  1.4491181  0.446020777 -0.25751187  1.66474
32
## [6,] -1.1547775 -0.5263678 -0.6187590 -0.068258525  0.08416854  2.03340
67
##              PC7        PC8         PC9
## [1,]  0.07927102 -1.0187599 -0.29485698
## [2,] -0.32898494 -1.0543512  0.07111559
## [3,]  0.50830477 -0.1184211  0.49628723
## [4,]  0.34139370  0.4354578 -0.46122443
## [5,]  1.27443050 -1.4171480 -0.98520328
## [6,] -1.62895108  0.1303434  1.02821475
```

```r
summary(pr.comp)
```

```
## Importance of components:
##                           PC1    PC2    PC3    PC4    PC5    PC6    PC7
PC8
## Standard deviation     1.1032 1.0420 1.0295 1.0070 0.9978 0.9800 0.9696
0.94216
## Proportion of Variance 0.1352 0.1206 0.1178 0.1127 0.1106 0.1067 0.1045
0.09863
## Cumulative Proportion  0.1352 0.2559 0.3736 0.4863 0.5969 0.7036 0.8081
0.90672
##                            PC9
## Standard deviation     0.91624
## Proportion of Variance 0.09328
## Cumulative Proportion  1.00000
```

```
(varience_pca <- (pr.comp$sdev)^2)
```

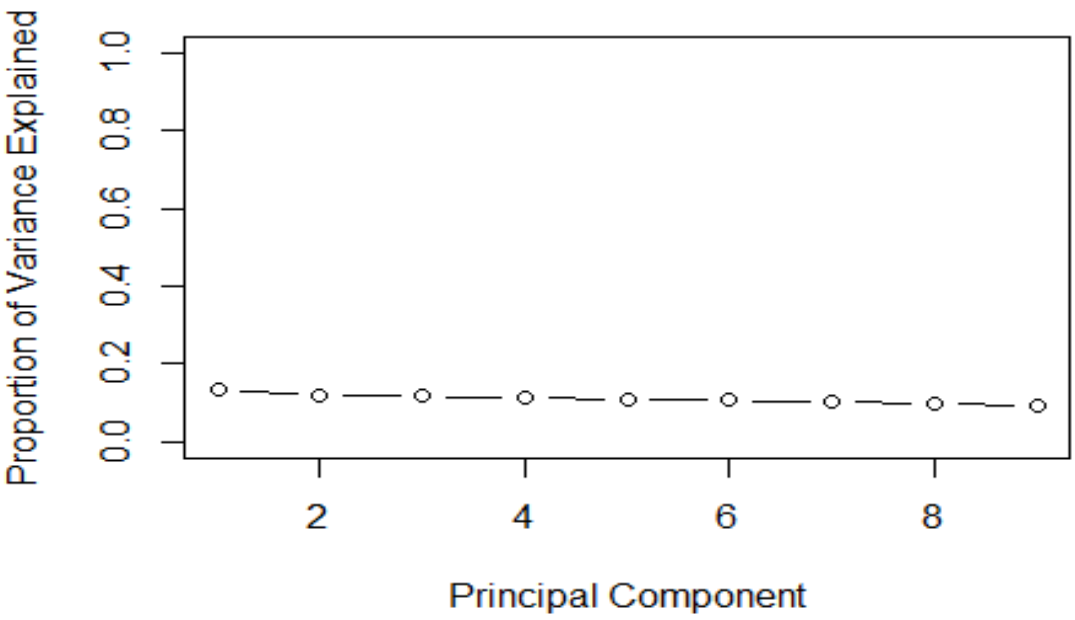
```
## [1] 1.2170936 1.0857084 1.0599538 1.0140727 0.9955516 0.9604195 0.94003
22
## [8] 0.8876644 0.8395036
```

```
(varience_comp <- varience_pca/sum(varience_pca))
```

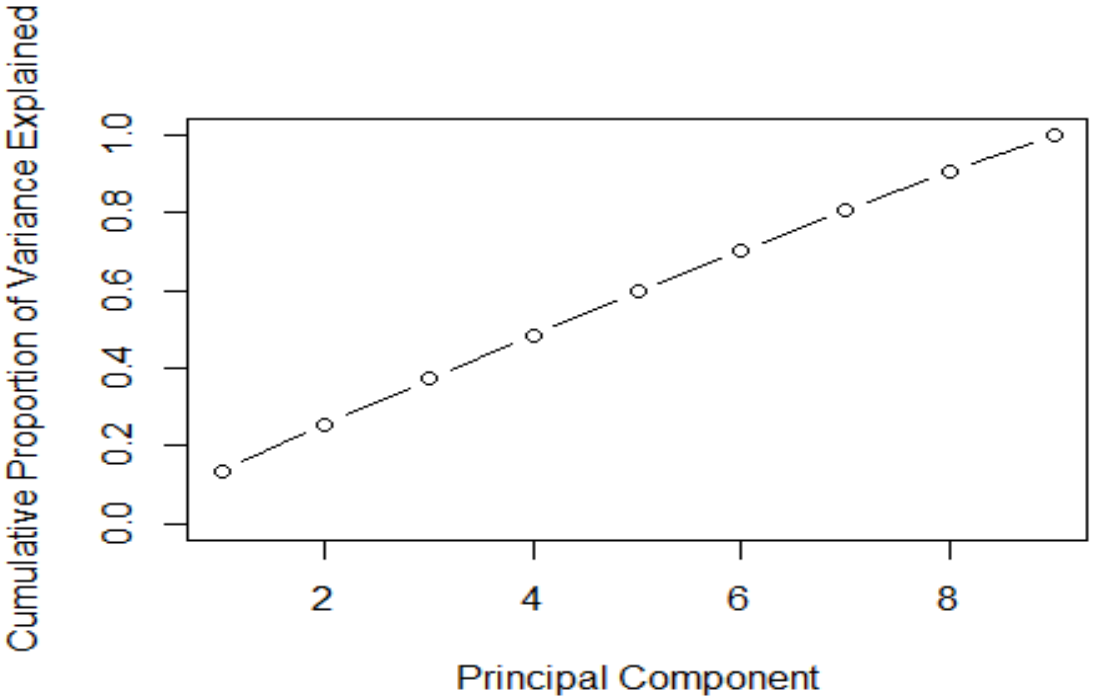
```
## [1] 0.13523263 0.12063426 0.11777265 0.11267475 0.11061684 0.10671328 0
.10444803
## [8] 0.09862938 0.09327818
```

#This gives the propotion of variance explained by each principal component.

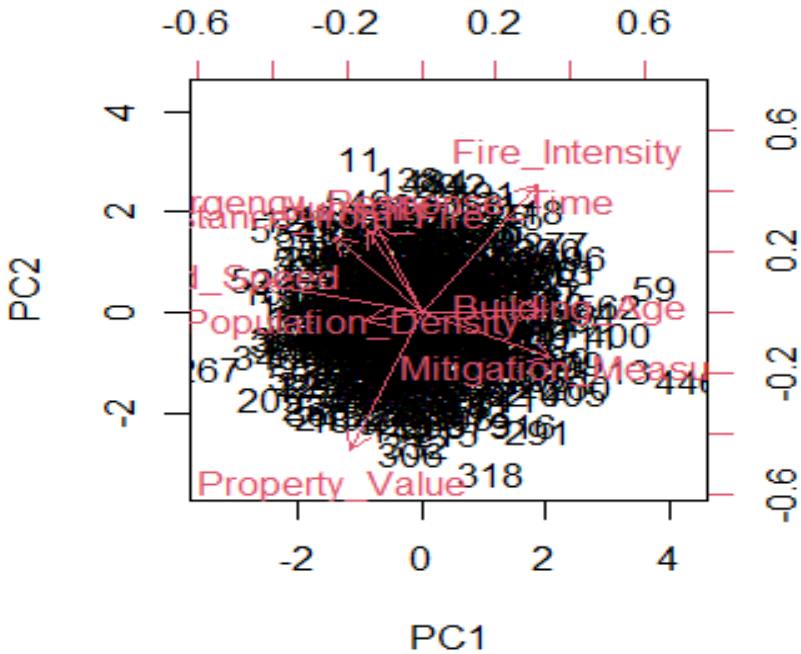
```
plot(varience_comp,xlab="Principal Component",ylab="Proportion of Variance
Explained",
ylim = c(0,1), type = 'b')
```



```
 plot(cumsum(varience_comp),xlab="Principal Component",ylab="Cumulative Pr
oportion of Variance Explained",
 ylim = c(0,1), type = 'b')
```




```
biplot(pr.comp, scale = 0)
```



K means Clustering

#Lets directly specify the number of clusters 2 and 3 to test.

```
set.seed(5)
kmeans_out <- kmeans(bushfire_cluster[, 1:9], 2, nstart = 20)

#3 clusters
kmeans_out2 <- kmeans(bushfire_cluster[, 1:9], 3, nstart = 20)

kmeans_out$cluster
```

```
##   [1] 1 2 1 2 2 1 2 2 2 1 2 1 2 2 1 2 1 1 1 2 2 2 2 2 2 2 2 2 1 1 2 2 2
2 1 1 2
##  [38] 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 2 1 2 1 1 2 2 1 2 1 1 1 2 1 2 2 2 1
2 2 2 1
##  [75] 1 1 1 2 1 1 1 2 1 2 1 1 2 2 2 1 1 2 1 2 1 1 2 1 1 1 2 1 1 2 1 2 2
2 1 1 2
## [112] 1 1 2 2 1 2 2 2 1 2 1 1 1 1 2 1 1 1 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1
2 1 1 1
## [149] 1 2 2 1 1 1 1 1 2 1 1 1 1 1 2 1 1 2 2 1 1 1 2 1 2 2 2 2 1 2 2 2 2
1 2 1 2
## [186] 1 1 2 2 2 1 1 2 2 2 1 1 2 1 2 1 2 2 2 1 2 1 1 1 1 1 1 1 2 1 2 1 1
2 2 1 2
## [223] 2 2 1 1 1 2 2 2 2 1 2 1 1 1 1 2 1 2 2 2 1 2 1 2 1 2 2 2 1 1 2 1 2
2 1 1 1
## [260] 2 2 2 2 2 2 2 1 1 1 1 2 1 1 1 2 2 2 1 1 2 2 1 1 1 1 1 1 1 2 1 1
1 2 2 2
## [297] 2 1 1 2 2 1 2 2 2 1 1 1 1 1 1 1 2 1 1 2 2 1 1 2 2 1 1 2 1 1 2 1 1
2 2 1 2
## [334] 2 1 1 2 2 2 2 1 1 2 1 1 1 2 1 1 2 1 2 2 1 2 2 2 1 1 2 1 1 2 1 1 2
1 1 1 1
## [371] 1 1 2 1 2 2 2 1 1 2 2 2 1 2 2 2 1 1 1 1 2 1 2 2 1 1 1 1 1 2 2 1 2
2 1 1 2
## [408] 1 1 2 1 2 1 1 1 1 2 1 1 1 2 2 2 1 2 2 1 1 1 2 2 2 1 2 1 2 1 1 1 1
1 2 2 1
## [445] 2 2 2 1 1 2 2 1 1 1 1 2 2 2 1 1 2 1 1 1 2 1 1 2 1 2 1 2 1 2 2 1 2
1 1 2 1
## [482] 2 1 2 2 1 1 2 1 2 2 1 1 2 1 2 1 1 2 2 1 1 1 1 1 1 2 1 2 1 1 1 2 1
2 1 1 2
## [519] 1 2 1 1 1 1 2 2 2 1 2 1 2 2 1 2 1 2 1 2 2 1 2 1 2 1 2 2 1 2 2 1 1
1 2 2 2
## [556] 2 2 2 1 1 1 2 2 1 2 2 1 2 2 2 2 2 2 1 2 2 1 1 2 1 2 2 2 1 2 1 1 2
2 2 2 1
## [593] 2 1 1 2 1 1 2 1
```

```
kmeans_out2$cluster
```

```
##   [1] 1 3 2 3 3 1 2 3 2 2 3 2 3 2 1 3 1 1 2 3 3 3 2 3 3 3 2 2 1 1 3 3 3
3 1 2 3
##  [38] 1 1 1 1 2 2 2 1 1 1 2 1 3 1 2 3 1 2 1 1 3 3 2 3 1 2 1 3 2 3 3 3 2
3 2 3 1
##  [75] 2 1 2 2 2 1 1 3 2 3 1 2 3 3 3 1 1 3 2 3 1 1 3 1 2 2 2 2 2 3 2 3 3
2 2 1 3
```

```
## [112] 1 1 3 3 1 2 3 2 2 2 1 1 1 2 3 1 1 1 3 2 3 3 3 2 3 3 3 3 2 1 2 1 1
3 1 1 1
## [149] 1 3 3 2 2 1 1 2 2 1 2 1 2 2 2 2 2 2 3 1 2 1 2 1 3 3 3 2 2 2 3 2 3
1 3 1 2
## [186] 2 1 2 3 3 1 1 3 2 3 2 2 3 1 2 1 3 2 2 2 3 2 1 1 1 2 1 1 3 1 3 2 2
3 3 1 3
## [223] 3 3 1 2 2 2 3 3 2 2 2 1 1 2 1 3 1 3 2 3 1 2 1 3 2 3 3 3 1 1 2 1 2
3 1 2 2
## [260] 3 3 3 3 3 2 2 2 2 1 2 3 1 1 1 3 3 3 2 1 2 3 1 2 1 1 2 1 1 1 3 1 1
1 3 3 3
## [297] 3 1 1 3 3 1 3 3 2 2 1 1 2 2 2 2 3 1 2 3 3 1 2 3 3 1 1 3 1 1 3 1 2
3 2 1 3
## [334] 3 1 2 3 2 2 3 2 1 2 1 2 2 3 2 1 2 1 3 2 1 2 3 3 2 1 3 1 1 3 1 2 3
1 1 2 2
## [371] 2 2 3 2 2 3 3 1 1 3 2 3 2 3 2 3 1 1 1 1 3 1 3 3 1 1 1 1 1 3 3 1 3
2 2 2 3
## [408] 1 2 3 1 3 1 2 1 1 3 1 2 1 2 2 2 1 3 2 1 2 1 3 3 2 2 3 1 2 1 2 1 1
1 2 3 1
## [445] 3 3 3 1 2 3 2 2 1 1 1 3 3 3 1 1 3 2 2 1 2 1 1 3 1 3 1 3 2 3 3 2 2
1 1 3 1
## [482] 3 1 3 3 1 1 3 2 3 3 1 2 3 1 3 1 1 3 3 2 2 1 1 2 1 2 2 3 2 2 1 3 2
3 1 2 3
## [519] 1 3 1 2 1 1 2 3 3 1 3 1 3 3 2 3 1 3 2 3 2 1 3 1 3 1 3 3 1 3 3 2 2
1 2 3 2
## [556] 2 3 2 2 1 1 3 3 1 2 3 2 3 2 3 2 3 2 2 3 3 2 1 2 2 3 3 3 2 3 1 1 3
3 2 2 1
## [593] 3 1 1 3 2 2 3 1
```

```
kmeans_out
```

```
## K-means clustering with 2 clusters of sizes 316, 284
##
## Cluster means:
##   Fire_Intensity Distance_from_Fire Building_Age Property_Value
## 1       5435.066           7.003734     24.68354       7.496266
## 2      11889.442           7.049718     25.10563       7.035282
##   Population_Density Emergency_Response_Time Mitigation_Measures Wind_S
peed
## 1           597.6171                14.92722            2.955696   24.9
7627
## 2           598.9613                15.01056            3.003521   25.0
1338
##   Humidity
## 1 49.50889
## 2 50.06246
##
## Clustering vector:
##   [1] 1 2 1 2 2 1 2 2 2 1 2 1 2 2 1 2 1 1 1 2 2 2 2 2 2 2 2 2 1 1 2 2 2
2 1 1 2
##  [38] 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 2 1 2 1 1 2 2 1 2 1 1 1 2 1 2 2 2 1
2 2 2 1
##  [75] 1 1 1 2 1 1 1 2 1 2 1 1 2 2 2 1 1 2 1 2 1 1 2 1 1 1 2 1 1 2 1 2 2
2 1 1 2
## [112] 1 1 2 2 1 2 2 2 1 2 1 1 1 1 2 1 1 1 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1
```

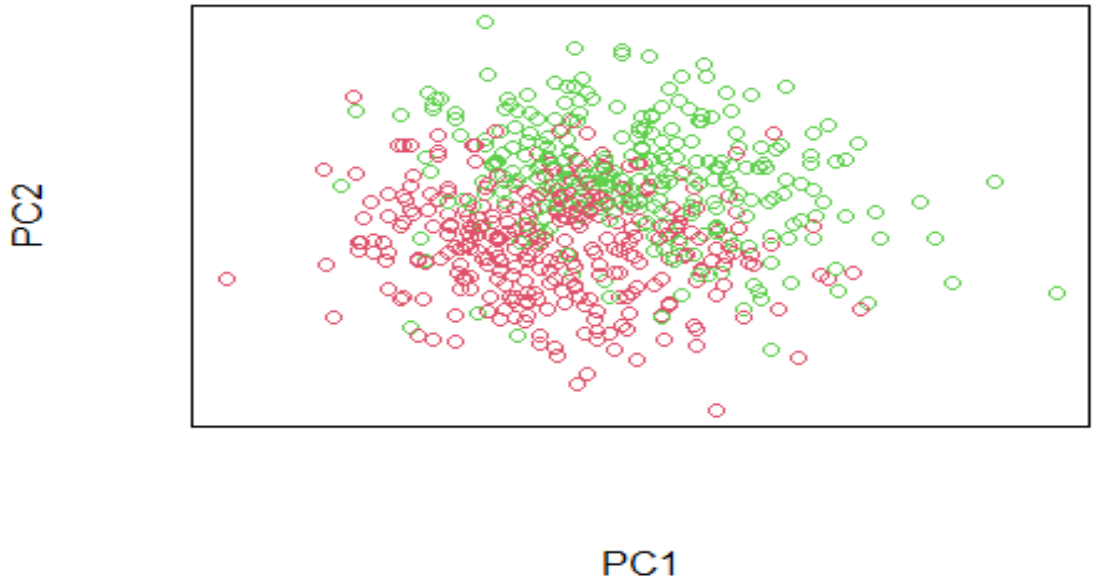
```
2 1 1 1
## [149] 1 2 2 1 1 1 1 1 2 1 1 1 1 1 2 1 1 2 2 1 1 1 2 1 2 2 2 2 1 2 2 2 2
1 2 1 2
## [186] 1 1 2 2 2 1 1 2 2 2 1 1 2 1 2 1 2 2 2 1 2 1 1 1 1 1 1 1 2 1 2 1 1
2 2 1 2
## [223] 2 2 1 1 1 2 2 2 2 1 2 1 1 1 1 2 1 2 2 2 1 2 1 2 1 2 2 2 1 1 2 1 2
2 1 1 1
## [260] 2 2 2 2 2 2 2 1 1 1 1 2 1 1 1 2 2 2 1 1 2 2 1 1 1 1 1 1 1 1 2 1 1
1 2 2 2
## [297] 2 1 1 2 2 1 2 2 2 1 1 1 1 1 1 1 2 1 1 2 2 1 1 2 2 1 1 2 1 1 2 1 1
2 2 1 2
## [334] 2 1 1 2 2 2 2 1 1 2 1 1 1 2 1 1 2 1 2 2 1 2 2 2 1 1 2 1 1 2 1 1 2
1 1 1 1
## [371] 1 1 2 1 2 2 2 1 1 2 2 2 1 2 2 2 1 1 1 1 2 1 2 2 1 1 1 1 1 2 2 1 2
2 1 1 2
## [408] 1 1 2 1 2 1 1 1 1 2 1 1 1 2 2 2 1 2 2 1 1 1 2 2 2 1 2 1 2 1 1 1 1
1 2 2 1
## [445] 2 2 2 1 1 2 2 1 1 1 1 2 2 2 1 1 2 1 1 1 2 1 1 2 1 2 1 2 1 2 2 1 2
1 1 2 1
## [482] 2 1 2 2 1 1 2 1 2 2 1 1 2 1 2 1 1 2 2 1 1 1 1 1 1 2 1 2 1 1 1 2 1
2 1 1 2
## [519] 1 2 1 1 1 1 2 2 2 1 2 1 2 2 1 2 1 2 1 2 2 1 2 1 2 1 2 2 1 2 2 1 1
1 2 2 2
## [556] 2 2 2 1 1 1 2 2 1 2 2 1 2 2 2 2 2 2 1 2 2 1 1 2 1 2 2 2 1 2 1 1 2
2 2 2 1
## [593] 2 1 1 2 1 1 2 1
##
## Within cluster sum of squares by cluster:
## [1] 1090005357  967451613
##  (between_SS / total_SS =  75.2 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.wi
thinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

```
kmeans_out2
```

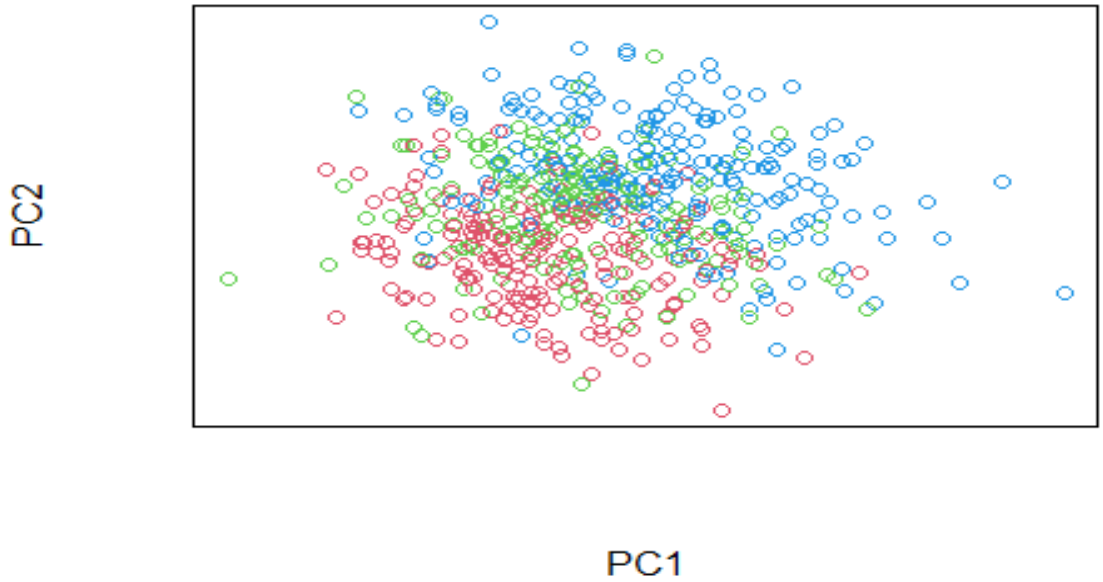
```
## K-means clustering with 3 clusters of sizes 197, 194, 209
##
## Cluster means:
##   Fire_Intensity Distance_from_Fire Building_Age Property_Value
## 1       4242.686           7.065431     24.64975       7.491168
## 2       8232.687           7.036804     24.69588       7.426443
## 3      12732.690           6.977368     25.27751       6.939474
##   Population_Density Emergency_Response_Time Mitigation_Measures Wind_S
peed
## 1           596.3553                14.61421            3.005076   25.2
3198
## 2           594.7784                15.48969            2.706186   24.9
9845
## 3           603.2679                14.81340            3.205742   24.7
6507
```

```
##   Humidity
## 1 47.97259
## 2 51.77041
## 3 49.61000
##
## Clustering vector:
##   [1] 1 3 2 3 3 1 2 3 2 2 3 2 3 2 1 3 1 1 2 3 3 3 2 3 3 3 2 2 1 1 3 3 3
3 1 2 3
##  [38] 1 1 1 1 2 2 2 1 1 1 2 1 3 1 2 3 1 2 1 1 3 3 2 3 1 2 1 3 2 3 3 3 2
3 2 3 1
##  [75] 2 1 2 2 2 1 1 3 2 3 1 2 3 3 3 1 1 3 2 3 1 1 3 1 2 2 2 2 2 3 2 3 3
2 2 1 3
## [112] 1 1 3 3 1 2 3 2 2 2 1 1 1 2 3 1 1 1 3 2 3 3 3 2 3 3 3 3 2 1 2 1 1
3 1 1 1
## [149] 1 3 3 2 2 1 1 2 2 1 2 1 2 2 2 2 2 2 3 1 2 1 2 1 3 3 3 2 2 2 3 2 3
1 3 1 2
## [186] 2 1 2 3 3 1 1 3 2 3 2 2 3 1 2 1 3 2 2 2 3 2 1 1 1 2 1 1 3 1 3 2 2
3 3 1 3
## [223] 3 3 1 2 2 2 3 3 2 2 2 1 1 2 1 3 1 3 2 3 1 2 1 3 2 3 3 3 1 1 2 1 2
3 1 2 2
## [260] 3 3 3 3 3 2 2 2 2 1 2 3 1 1 1 3 3 3 2 1 2 3 1 2 1 1 2 1 1 1 3 1 1
1 3 3 3
## [297] 3 1 1 3 3 1 3 3 2 2 1 1 2 2 2 2 3 1 2 3 3 1 2 3 3 1 1 3 1 1 3 1 2
3 2 1 3
## [334] 3 1 2 3 2 2 3 2 1 2 1 2 2 3 2 1 2 1 3 2 1 2 3 3 2 1 3 1 1 3 1 2 3
1 1 2 2
## [371] 2 2 3 2 2 3 3 1 1 3 2 3 2 3 2 3 1 1 1 1 3 1 3 3 1 1 1 1 1 3 3 1 3
2 2 2 3
## [408] 1 2 3 1 3 1 2 1 1 3 1 2 1 2 2 2 1 3 2 1 2 1 3 3 2 2 3 1 2 1 2 1 1
1 2 3 1
## [445] 3 3 3 1 2 3 2 2 1 1 1 3 3 3 1 1 3 2 2 1 2 1 1 3 1 3 1 3 2 3 3 2 2
1 1 3 1
## [482] 3 1 3 3 1 1 3 2 3 3 1 2 3 1 3 1 1 3 3 2 2 1 1 2 1 2 2 3 2 2 1 3 2
3 1 2 3
## [519] 1 3 1 2 1 1 2 3 3 1 3 1 3 3 2 3 1 3 2 3 2 1 3 1 3 1 3 3 1 3 3 2 2
1 2 3 2
## [556] 2 3 2 2 1 1 3 3 1 2 3 2 3 2 3 2 3 2 2 3 3 2 1 2 2 3 3 3 2 3 1 1 3
3 2 2 1
## [593] 3 1 1 3 2 2 3 1
##
## Within cluster sum of squares by cluster:
## [1] 290622002 286399109 382747287
##  (between_SS / total_SS =  88.4 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.wi
thinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

```
clusterPlot1 <- prcomp(bushfire_cluster[,1:9], scale = T)

plot(clusterPlot1$x[,1:4], col = fitted(kmeans_out, "classes")+1, xaxt = "
n", yaxt = "n")
```



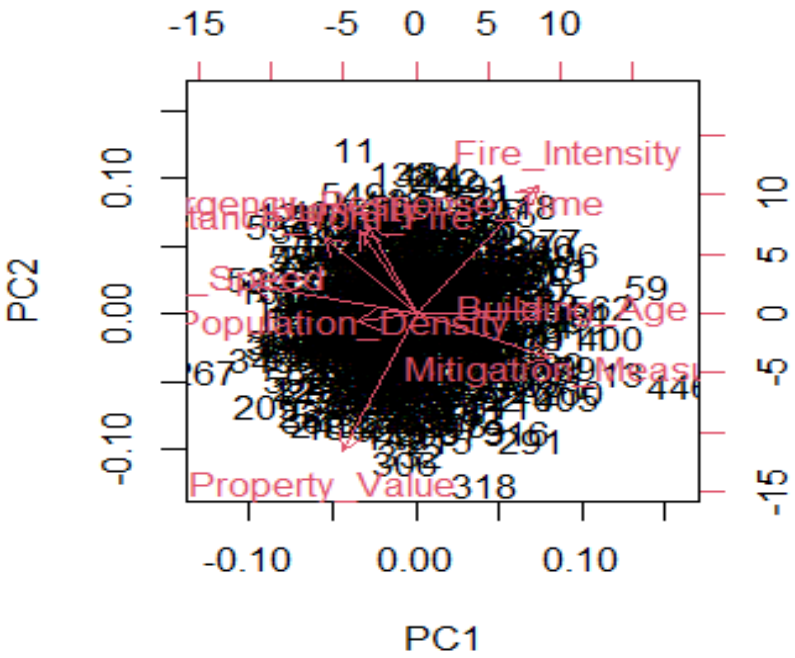

```
clusterPlot2 <- prcomp(bushfire_cluster[,1:9], scale = T)

plot(clusterPlot2$x[,1:6], col = fitted(kmeans_out2, "classes")+1, xaxt =
"n", yaxt = "n")
```




```
summary(clusterPlot1)
```

```
## Importance of components:
##                           PC1    PC2    PC3    PC4    PC5    PC6    PC7
PC8
## Standard deviation     1.1032 1.0420 1.0295 1.0070 0.9978 0.9800 0.9696
0.94216
## Proportion of Variance 0.1352 0.1206 0.1178 0.1127 0.1106 0.1067 0.1045
0.09863
## Cumulative Proportion  0.1352 0.2559 0.3736 0.4863 0.5969 0.7036 0.8081
0.90672
##                            PC9
## Standard deviation     0.91624
## Proportion of Variance 0.09328
## Cumulative Proportion  1.00000
```

```
biplot(clusterPlot1)
```




```
bushfire_data_scaled <- scale(bushfire_cluster)
```

Hierarchical Clustering

```
 hc.complete = hclust(dist(bushfire_data_scaled), method = "complete")
 hc.complete
```

```
##
## Call:
## hclust(d = dist(bushfire_data_scaled), method = "complete")
##
## Cluster method   : complete
## Distance         : euclidean
## Number of objects: 600
```

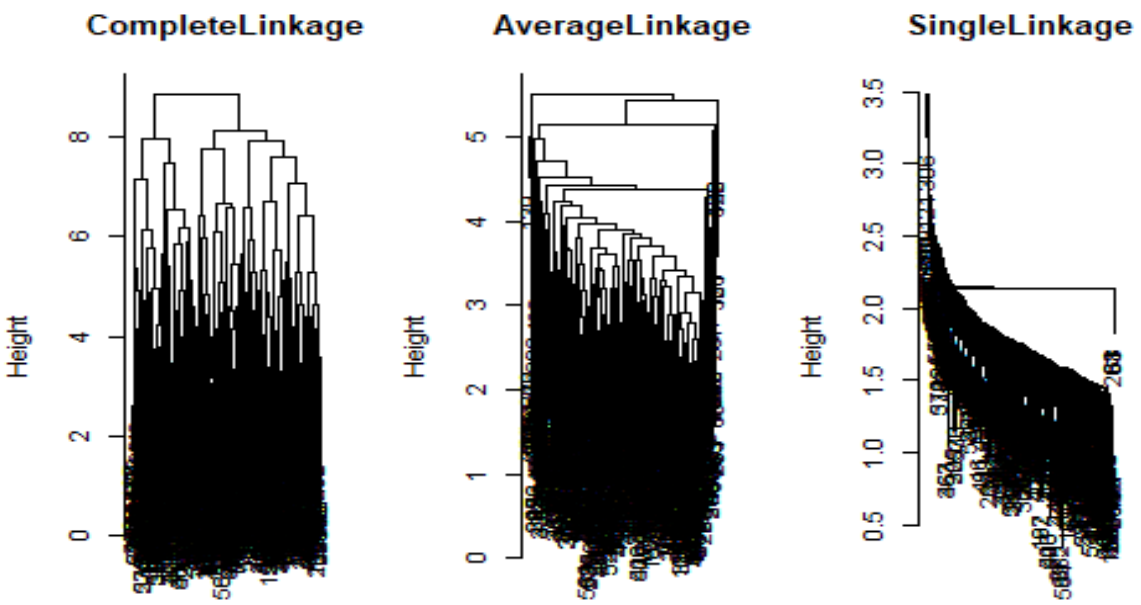
```r
hc.single=hclust(dist(bushfire_data_scaled),method = "single")
hc.single
```

```
##
## Call:
## hclust(d = dist(bushfire_data_scaled), method = "single")
##
## Cluster method   : single
## Distance         : euclidean
## Number of objects: 600
```

```r
hc.average = hclust(dist(bushfire_data_scaled), method = "average")
hc.average
```

```
##
## Call:
## hclust(d = dist(bushfire_data_scaled), method = "average")
##
## Cluster method   : average
## Distance         : euclidean
## Number of objects: 600
```

```r
par(mfrow=c(1,3))
plot(hc.complete,main="CompleteLinkage",xlab="",sub="",cex=0.9)
plot(hc.average,main="AverageLinkage",xlab="",sub="",cex=0.9)
plot(hc.single,main="SingleLinkage",xlab="",sub="",cex=0.9)
```




```r
cutree(hc.complete,2)
```

```
##   [1] 1 1 2 1 1 1 2 1 1 1 2 2 2 2 1 2 2 1 1 2 1 1 1 1 1 1 2 1 1 1 1 2 2
2 1 1 1
##  [38] 1 1 1 1 2 1 2 1 1 1 2 1 1 1 2 2 1 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1
```

```
1 1 1 1
##  [75] 1 2 1 1 1 1 1 1 2 1 1 1 1 2 1 1 2 2 1 2 1 1 1 1 2 2 1 1 2 2 1 2 1
1 2 1 1
## [112] 1 2 1 2 2 1 2 1 1 1 1 1 1 2 1 1 1 1 2 2 2 1 2 1 1 2 2 2 1 1 1 1 1
2 1 2 1
## [149] 2 2 1 2 1 1 1 1 2 2 2 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 2 1 2 1 1
1 1 1 1
## [186] 1 1 2 2 2 2 1 1 1 2 2 1 2 1 1 2 2 2 1 1 2 2 2 1 2 1 1 1 2 2 2 1 1
2 1 2 1
## [223] 2 2 1 1 1 2 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 2 1 2 2 1 1 1 2 1 1 1 1
2 1 2 2
## [260] 1 1 2 1 1 2 1 1 1 1 2 1 1 1 1 2 1 1 1 2 1 1 1 1 1 2 1 1 1 2 1 2 1
1 1 2 2
## [297] 2 1 1 1 2 1 1 1 1 2 1 1 1 2 1 2 2 1 1 2 2 2 2 1 2 1 1 2 1 1 1 1 1
1 1 2 1
## [334] 1 2 2 1 1 2 1 2 1 1 1 2 1 2 1 1 2 1 2 1 2 1 2 2 1 1 1 1 1 2 1 1 2
1 2 1 2
## [371] 2 1 2 1 2 1 2 1 1 1 1 2 1 2 1 2 2 1 1 1 2 1 1 2 2 2 1 1 2 1 1 2 1
2 1 1 1
## [408] 1 1 1 1 1 2 1 1 1 1 2 1 1 1 1 2 2 2 1 1 1 1 2 2 2 1 2 1 1 1 1 1 1
1 1 2 1
## [445] 1 2 1 1 1 2 1 1 1 1 1 2 1 2 1 1 1 1 1 1 1 1 1 2 2 1 1 2 1 2 1 1 1
1 1 2 2
## [482] 1 2 1 2 1 1 1 1 2 2 1 2 2 1 2 2 1 2 2 1 1 2 1 1 2 1 1 1 1 2 2 1 1
2 1 2 1
## [519] 2 2 2 1 1 1 2 2 2 1 1 1 2 2 1 1 2 2 1 2 2 1 2 1 2 1 1 2 1 1 2 1 1
1 1 2 1
## [556] 1 1 1 1 1 2 2 1 2 1 1 1 2 1 1 1 2 1 2 2 1 2 1 2 2 1 2 1 1 1 1 1 2
1 1 2 1
## [593] 2 1 1 2 1 2 1 1
```

```
cutree(hc.average,2)
```

```
##   [1] 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1
1 1 1 1
##  [38] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
##  [75] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [112] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1
1 1 1 1
## [149] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [186] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [223] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [260] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [297] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [334] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [371] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

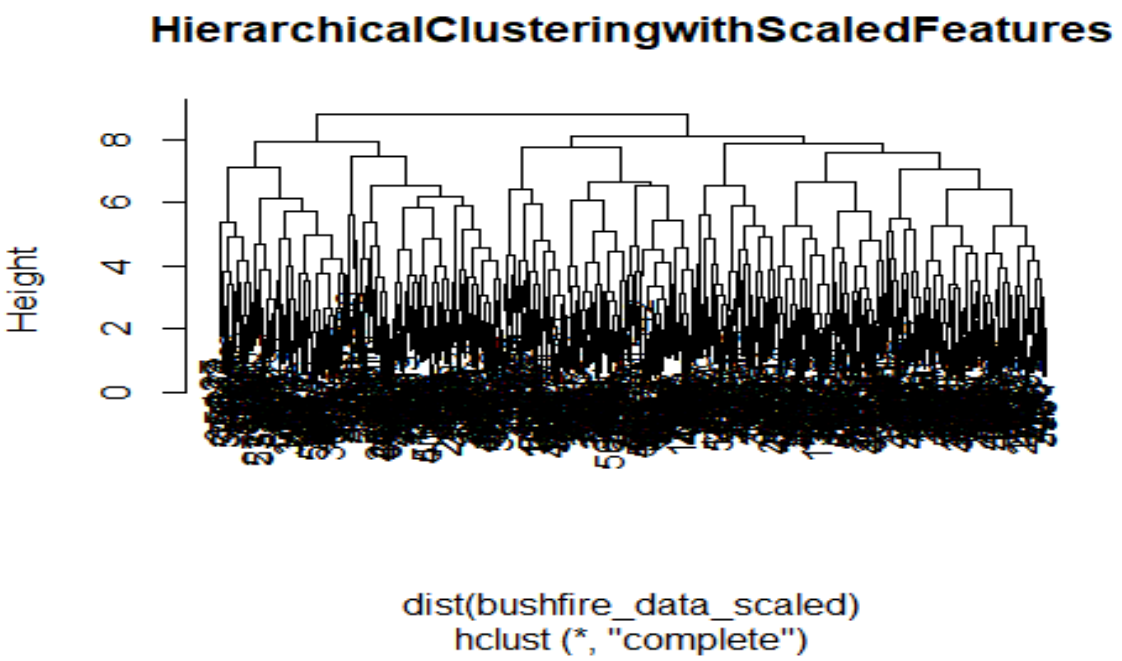
```
1 1 1 1
## [408] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [445] 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [482] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [519] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 2 1
## [556] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [593] 1 1 1 1 1 1 1 1
```

```
cutree(hc.single,2)
```

```
##   [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
##  [38] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
##  [75] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [112] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [149] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [186] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [223] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [260] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [297] 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [334] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [371] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [408] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [445] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [482] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [519] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [556] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [593] 1 1 1 1 1 1 1 1
```

```
 cutree(hc.single,9)
```

```
##   [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1
1 1 1 1
##  [38] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
```

```
##  [75] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [112] 1 1 1 1 1 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 4 1 1 1 1 1
1 1 1 1
## [149] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [186] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [223] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [260] 1 1 1 1 1 1 1 5 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [297] 1 1 1 1 1 1 1 1 1 6 1 1 1 1 1 1 1 1 1 7 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [334] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [371] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [408] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [445] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [482] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [519] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 8 1 1 1 1 1 1 1 1 1 1
1 1 1 1
## [556] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
9 1 1 1
## [593] 1 1 1 1 1 1 1 1
```

```r
plot(hclust(dist(bushfire_data_scaled),method="complete"),main="Hierarchic
alClusteringwithScaledFeatures")
```